ФСП федерация спортивного программирования России

ФСП Job Board

сопроводительная документация

Цифровая платформа подбора IT-специалистов с верифицированным профилем достижений участника ФСП. Обратная механика: категоризация по адаптивному тесту, выход на контакт по инициативе работодателя.

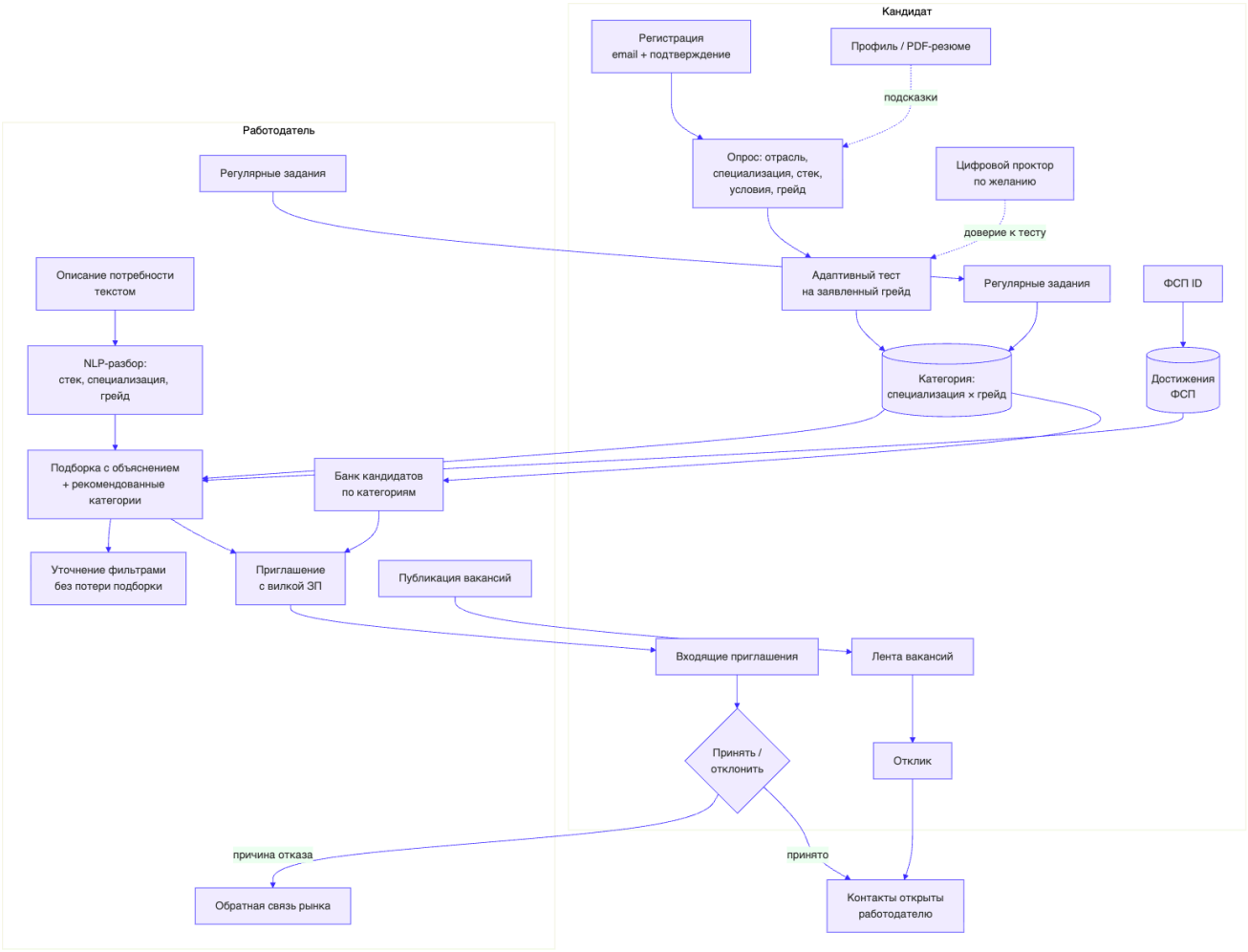
1. Архитектура решения
2. Механика тестирования
3. Механика подбора
4. Валидация
5. Интеграция с ФСП
6. API
7. Компоненты
8. Развёртывание
9. Концепции развития
10. Живой пилот

Версия от 09.10.2026 · документация собрана из репозитория (docs/*.md)

Архитектура решения

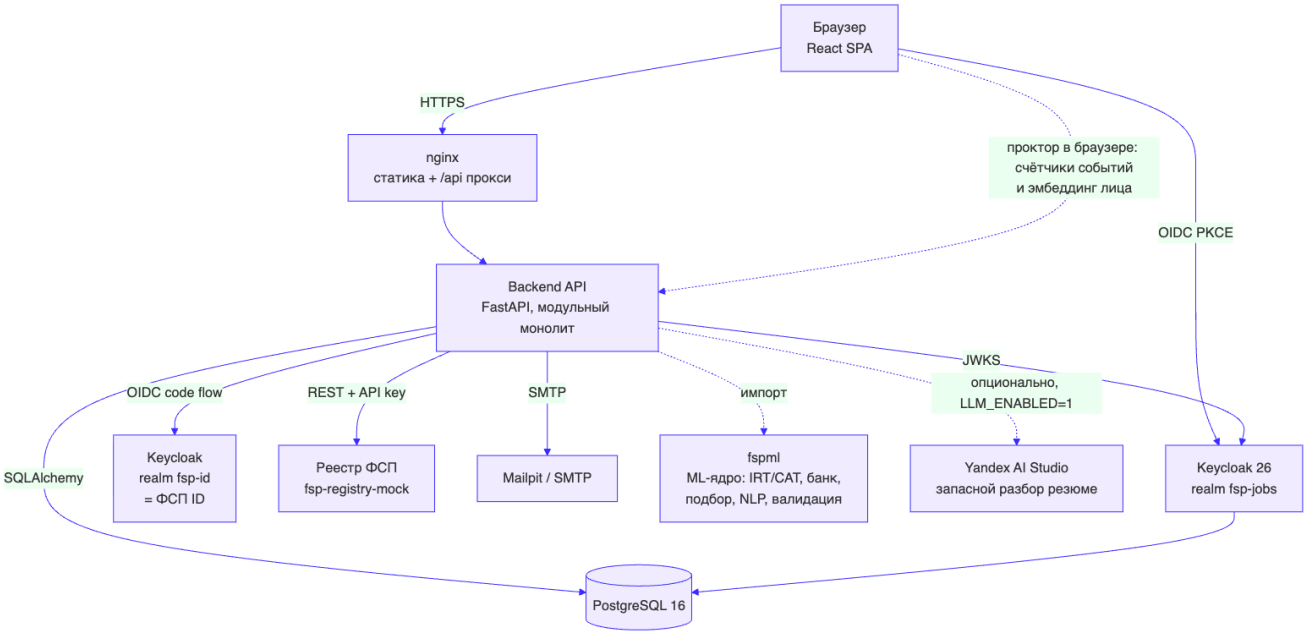
1. Функциональная архитектура

Платформа соединяет две роли напрямую, без площадок-посредников. Смысловой центр — **обратная механика**: категорию кандидата определяет тест, работодатель сам находит категорию и выходит на контакт.



Сценарий ТЗ	Где реализован
Регистрация по email с подтверждением	Keycloak realm `fsp-jobs` (verifyEmail), письма — SMTP (Mailpit в демо)
Профиль вручную или из PDF, стандартизированный PDF-профиль	`candidates` (`/api/candidate/profile` , `/resume` , `/profile.pdf`), `fspml.nlp.resume`
Опрос → грейд → тест → категория	`candidates/survey` , `assessment` , `fspml.assessment`
Регулярные задания	`tasks`
Тест на грейд выше/ниже с ограничением частоты	`fspml.assessment.grade_policy`
Приглашения: принять/отклонить, статусы	`invitations`
Вакансии и отклики	`vacancies`
Приватность, согласия, ФСП ID	`users` (согласия 152-ФЗ), `candidates/privacy` , `fsp`
Компания, потребность, подборка с обоснованием, банк с фильтрами	`employers` , `fspml.matching` , `fspml.nlp.vacancy`
Контакты скрыты до принятия/отклика	`invitations/access.py` (единая точка правила)

2. Компонентная архитектура



Компонент	Технологии	Ответственность
`frontend/`	React 19, TypeScript, Vite, Tailwind CSS 4, TanStack Query, keycloak-js, Human (TF.js)	Два кабинета, публичные страницы механик, цифровой проктор в браузере; адаптивная вёрстка
`backend/`	Python 3.12, FastAPI, SQLAlchemy 2, Alembic, Pydantic 2, fpdf2, cryptography	API бизнес-логики, правила доступа, оркестрация ML, сверка лица 1:1 (`BiometricProvider`)
`ml/` (`fspml`)	numpy, scipy, scikit-learn, lightgbm, rapidfuzz, pdfplumber, pandas	Тестирование (IRT/CAT, 303 семейства заданий), подбор, разбор текстов, валидация
`fsp-registry-mock/`	FastAPI	Имитация реестра ФСП с предложенной схемой данных
`keycloak/`	Keycloak 26 (realm-export)	Аутентификация, подтверждение email, ФСП ID (демо-realm)
PostgreSQL 16	—	Реляционное хранилище (18 таблиц) + БД Keycloak

Модули бэкенда (модульный монолит)

Каждый модуль — `models.py` (таблицы), `router.py` (API), при необходимости `service.py` . Модули связаны только через явные импорты сервисных функций; бизнес-правила вынесены в одну точку (например, видимость контактов — `invitations/access.py` , переходы грейда — `fspml.assessment.grade_policy`).

Модуль	Таблицы	Назначение
`users`	`users` , `consents` , `audit_log`	Пользователь (sub из IdP), роль, согласия 152-ФЗ с версиями, журнал доступа к ПДн
`catalog`	—	Справочники из `fspml/data/catalog.yaml`
`candidates`	`candidate_profiles`	Профиль, опрос, разбор резюме, PDF, приватность
`assessment`	`attempts` , `attempt_items` , `family_stats`	Попытки теста, выданные задания (без текста — HMAC-seed), мониторинг утечек и перекалибровка
`employers`	`companies` , `searches` , `search_versions` , `shortlist`	Компания, подборки с версиями фильтров, банк кандидатов
`invitations`	`invitations`	Приглашения, статусы, квоты, обратная связь рынка, webhook ATS
`vacancies`	`vacancies` , `applications`	Вакансии и отклики
`tasks`	`regular_tasks` , `task_assignments`	Регулярные задания
`fsp`	(поля профиля)	Привязка ФСП ID, синхронизация реестра (фоновая задача)
`biometrics`	`biometric_templates` , `attempt_snapshots`	Цифровой проктор: эталон лица (AES-256-GCM, `local_demo` ; в production — ЕБС), кадр с теста — работодателю только после принятого приглашения или отклика и при согласии (`/api/employer/candidates/{id}/test-photo`)

Принципы

- **ML-ядро без инфраструктуры.** `fspml` не знает о БД и вебе: чистые функции и dataclass-ы. Его тестируют (более 1000 тестов) и валидируют (`make validate`) независимо; бэкенд — тонкий слой.
- **По умолчанию — без внешних сервисов.** Персональные данные не покидают контур (152-ФЗ): разбор текстов — словарь + нечёткий поиск, эмбеддинги — локальная модель или TF-IDF, тесты — детерминированные генераторы, проктор работает в браузере кандидата. Опционально (`LLM_ENABLED=1`) — Yandex AI Studio в контуре РФ как запасной разбор опыта в резюме, с отключённым логированием запросов.

- **Состояние теста — журнал ответов.** Попытка восстанавливается из `attempt_items`; текст задания не хранится, а воспроизводится по HMAC(секрет, попытка, позиция) — для апелляций и аудита.
- **Масштабирование.** API без состояния (горизонтально), ранжирование — SQL-фильтр по категории (сотни–тысячи строк) + векторизованный скоринг; путь роста — pgvector/ANN для семантики и материализованные агрегаты категорий.
- **Ресурсы.** Весь стек ≈ 1.5–2 ГБ RAM (Keycloak — основной потребитель), CPU-only. Подходит недорогой VPS 2–4 vCPU / 4–8 ГБ.

3. Безопасность и 152-ФЗ

Требование	Реализация
Согласие на обработку и публикацию	Обязательное согласие при выборе роли; версия и время хранятся (`consents`); отзыв скрывает профиль (`hidden`)
Разграничение прав по ролям	Роль в БД, зависимости `require_role`; тесты на 401/403/404 (`backend/tests/test_flow.py`)
Защита ПДн	Обезличенные карточки (псевдоним), контакты только после принятия/отклика, журнал раскрытия (`audit_log`), скрытие от отраслей/компаний
Пароли и токены	Пароли хранит только Keycloak (PBKDF2), бэкенд проверяет подпись JWT по JWKS, PKCE для SPA, refresh — в Keycloak-сессии
Честность теста	Ответы только на сервере, ограничение времени проверяется на сервере, полноэкранный режим, фиксация смены вкладки, второго монитора, RDP/ВМ и автоматизации браузера; брошенная попытка = неудачная; цифровой проктор по желанию — видео не записывается, на сервер уходят счётчики событий и эмбеддинг лица
Минимизация	Ожидания по ЗП не раскрываются работодателю — только индикатор совместимости вилки

Механика тестирования и категоризации

> Код: `ml/fspml/assessment/` (движок), `backend/app/modules/assessment/` (API). Результаты проверки — validation.md.

1. Задача и требования ТЗ

Категория кандидата = **специализация × грейд**. Специализацию кандидат выбирает в обязательном опросе, грейд заявляет сам, а подтверждает — тестом. Резюме на категорию не влияет.

Требование ТЗ	Решение
Единый набор заданий не подходит — задания разойдутся	Задание — **шаблон-генератор**: у каждого кандидата свой вариант; утёкший вариант бесполезен
Генерация нейросетью не гарантирует сопоставимую сложность	Сложность — свойство **шаблона** (семейства), варианты изоморфны; правильный ответ вычисляется, а не генерируется
Стабильный грейд при повторе	Классификационный адаптивный тест (CAT), правила смены грейда, проверка тест–ретест в валидации
Дискриминативность	Модель IRT 3PL, отбор по информации Фишера, мониторинг точечно-бисериальной корреляции
Тест на уровень ниже/выше, ограничение частоты	Машина состояний грейда (`grade_policy.py`)
Показать, как из вакансий формируются тесты	NLP-разбор вакансии → blueprint (доли навыков) → CAT; страница «Тест из вакансии» и `POST /api/assessment/preview`

2. Банк: параметризованные семейства заданий

Семейство (item family, подход *automatic item generation* / *item cloning*) — функция `rng → экземпляр`. Она рандомизирует параметры (числа, данные таблиц, имена, фрагменты кода, мутации) и **вычисляет правильный ответ**: Python-сниппеты исполняются, SQL выполняется в in-memory SQLite, алгоритмические задачи решаются эталонной реализацией. Ответы для JavaScript, Java, Go, C#, PHP и Vue считает Python-модель семантики языка; код заданий на Java, Go, C# и PHP — полная программа, в CI её исполняют настоящие JVM, Go, .NET и PHP, JavaScript — Node.js.

```
@family("sql.left_join", "sql", 3, "LEFT JOIN и строки без пары")
def _(rng):
    users = ...; orders = ...                    # случайные данные
    q = rng.choice([...три вида запроса...])
    ans = run_sql(setup, q)[0][0]                # ответ — результат реального запроса
    return number(prompt, ans, code=q, code_lang="sql")
```

Состав банка (`ml/fspml/assessment/bank/`): **303 семейства** по 31 навыку (в том числе направления DS: компьютерное зрение, NLP, рекомендательные системы, временные ряды; языки и фреймворки стека: Python, JavaScript, TypeScript, Java, Go, C#, PHP, React, Vue) и 7 специализациям, уровни сложности 1–5. Форматы — ввод числа/строки (предпочтительно: угадывание ≈ 0, утёкший ключ не подходит к новому варианту), выбор одного/нескольких вариантов с дистракторами из типичных ошибок, упорядочивание. Медианно семейство даёт ~166 уникальных вариантов из 200 генераций.

Автопроверки в CI (`ml/tests/test_bank.py` , 917 проверок): правильный ответ принимается проверкой; неверный — нет; варианты ответа не дублируются; экземпляр детерминирован по seed; у вычислимых семейств ответ меняется между экземплярами; у каждого навыка есть семейства ≥ 3 уровней; вывод заданий совпадает с Node.js, JVM, Go, .NET и PHP.

3. Модель измерения: IRT 3PL

```
P(верно | θ) = c + (1 - c) / (1 + exp(-a·(θ - b)))
```

- **b** — сложность семейства. Начальное значение — по экспертному уровню: L1…L5 → –1.75, –0.75, 0.25, 1.15, 2.0 (согласовано с порогами грейдов).
- **a** — дискриминативность, по умолчанию 1.3.
- **c** — угадывание, **фиксируется форматом** (1/k для выбора из k вариантов, 0 для ввода) и не оценивается — модель устойчива при малом числе ответов.
- **θ** — способность кандидата на шкале специализации (средний кандидат — 0). Оценка **EAP** на сетке [–4, 4] с приором N(0, 1): устойчива при всех верных/неверных ответах и сразу даёт P(θ > порог).

Пороги грейдов (`catalog.yaml`): Intern –2.25 · Junior –1.25 · Middle –0.25 · Senior 0.75 · Lead 1.6.

4. Классификационный адаптивный тест (CAT)

Цель теста — не измерить θ «вообще», а **решить, выше ли способность порога заявленного грейда**. Отсюда алгоритм (`cat.py`):

1. **Балансировка содержания.** Следующий навык — с наибольшим дефицитом относительно персонального состава теста (§4.1). Когда до минимальной длины остаётся столько заданий, сколько не хватает до покрытия, выбор ограничивается навыками, у которых ещё меньше 2 заданий.

2. **Точка нацеливания.** Первые 3 задания — в пороге заявленного грейда; далее — в том из порогов {заявленный, следующий}, решение по которому сейчас наиболее неопределённо (P ближе к 0.5). Так тест параллельно собирает данные для предложения повысить грейд.
3. **Выбор задания** — максимум информации Фишера в точке нацеливания, случайно из топ-4 (*randomesque*) — контроль экспозиции; есть потолок частоты показа семейства по данным за 30 дней. - **Без повторов внутри попытки:** семейство не выдаётся дважды; семейства одной *группы* (один и тот же вопрос в разной форме: `GROUP BY` в SQL и `groupby` в pandas, размножение строк при `JOIN` и `merge`, A/B-тест в статистике и в метриках) — тоже. - **Без повторов между попытками:** при старте попытки в неё записываются семейства, которые кандидат видел по этой специализации за 180 дней (`attempts.seen_families`). Внутри навыка они выбираются только тогда, когда новых не осталось, — тест не обрывается. Так тест на ступень ниже после неудачи, повышение и ретест идут на *параллельной форме*, а не на тех же шаблонах с другими числами. Эффект в валидации: доля повторов в следующей попытке онбординга снизилась с 54% до 15%, на ретесте — с 62% до 28% при той же точности грейда (70,4%) (validation.md).
4. **Остановка**: не меньше 10 заданий, каждый навык состава получил не меньше 2 заданий (иначе его подоценка — «нет данных», а подтверждённые тестом навыки — главный сигнал подбора) и $P(\theta > \text{порог}) \geq 0.95$ или ≤ 0.05; не больше 25 заданий. В среднем — 18 заданий.
5. **Решение**: подтверждён при $P(\theta > \text{порог}) \geq$ **0.6** (первичное подтверждение) или $\geq$ **0.75** (повышение уже подтверждённого грейда); предложение пройти тест выше — при $P(\theta > \text{порог следующего}) \geq 0.8$. Пороги выбраны по ROC-анализу в валидации.
6. **Подоценки навыков** — иерархическая EAP: приор навыка $N(\theta$ без этого навыка, $0.6^2)$. При 2–4 заданиях на навык оценка сжимается к общему уровню, поэтому статус («подтверждено / частично / не подтверждено / нет данных») не скачет при повторе.

4.1 Состав теста: ядро специализации + модули стека

Было. Состав теста был общим для специализации, а заявленный стек выбирал только язык backend-теста — первый совпавший из [python, javascript], а при отсутствии совпадения подставлялся Python. В итоге задания выходили за пределы профиля: Java-, Go-, C#-, PHP-разработчики (по архетипам рынка — ~60% backend) получали 20% теста по Python, Vue- и Angular-разработчики — задания по React, Linux-администраторы — по Kubernetes, QA на Java — по Python. В среднем по архетипам вне заявленного стека оказывалось 15–37% теста (backend 37%, DS 35%). Такие задания измеряют не грейд, а знакомство с чужим стеком, и систематически занижают оценку; к тому же 5%-навыки получали по одному заданию и оставались «нет данных».

Стало (`catalog.yaml` → `core` / `modules` , `assessment/blueprint.py`):

- **Ядро** — фундамент специализации, у всех кандидатов категории (сопоставимость грейдов, языконезависимые якоря шкалы). Отобрано по покрытию архетипов стека (`stack_archetypes.yaml`) плюс общепринятый фундамент: Backend — алгоритмы, SQL, HTTP, архитектура; Frontend — JavaScript, HTML/CSS, HTTP, алгоритмы; Mobile — мобильные платформы, HTTP, алгоритмы, архитектура; DS — ML, статистика, Python; QA — тест-дизайн, HTTP; DevOps — Linux, Docker, CI/CD, сети; аналитик — требования, SQL.
- **Модули** — навыки, которые подтверждают заявленные технологии (Django и FastAPI → Python, Spring → Java, Gin → Go, .NET → C#, Laravel → PHP, React → React, Vue → Vue, Appium → мобильные платформы): до 3 модулей, 20% теста на модуль, не больше 40% на все (ядро — не меньше 60%); внутри — пропорционально √числа технологий (язык с фреймворками весит больше Git).
- **Технология без заданий** (например, Rust, Ruby, Scala, Angular) ничем не подменяется: тест опирается на ядро, в профиле она остаётся «заявленной». Кандидат видит свой состав до старта: ядро, модули с технологиями-источниками и список технологий без заданий (`GET /api/assessment/eligibility` → `blueprint`).

Направления внутри специализации. У DS есть направления — табличные данные, компьютерное зрение, NLP, рекомендательные системы, временные ряды (`catalog.yaml` → `directions`). Кандидат выбирает до двух в опросе. Навык направления входит в тест модулем с приоритетом над стеком (весит как две заявленные технологии): CV-инженер получает задания по IoU, NMS, свёрткам и валидации на видео, даже если не перечислил OpenCV. Лимиты модулей прежние, ядро — не меньше 60%, поэтому грейды в категории остаются сопоставимыми. Механизм общий и может быть включён для других специализаций.

Тест подтверждает то, что кандидат заявил, и не проверяет того, чего он не заявлял; накрутка стека при этом не выгодна — заявленная технология превращается в модуль и проверяется.

Проверка (validation.md §2.3): на 600 кандидатах с реалистичными стеками, когда стек не влияет на ответы, новый состав сопоставим с прежним по точности (67.2% против 67.8%, в пределах шума) и стабильнее при повторе (72.0% против 67.7%); когда влияет (способность в чужом стеке на 1 SD ниже общей), прежний состав занижает грейд у затронутых кандидатов в 36.7% случаев (смещение θ –0.14), новый — в 27.1% (–0.02). Доля заявленных технологий, по которым у работодателя есть данные теста, растёт с 77% до 94%, и для подбора это заметно: подоценки теста при прежнем составе ухудшали выдачу (–0.023 NDCG@10), при новом — улучшают (+0.067, validation.md §4). Цена — +0.4 задания на попытку.

5. Правила грейда (машина состояний)

Ситуация	Правило
Онбординг (грейд ещё не подтверждён)	Тест на любой грейд. После неудачи на грейде g: ниже g — сразу, g и выше — через 14 дней
Первое подтверждение	Не считается сменой; сразу доступно повышение на одну ступень
Повышение	Только на одну ступень; смена грейда не чаще раза в **90 дней**
Неудача на более высоком уровне	Текущий грейд сохраняется — **грейд не понижается** по результату теста
Повтор на свой грейд (обновить результат)	Раз в 14 дней
Брошенная попытка	Засчитывается как неудачная (защита от «перезапуска до удачного набора»)
Смена специализации	Новая категория: прежний результат не переносится

6. Тест из описания вакансии

`fspml.nlp.vacancy.parse_need` выделяет технологии с важностью (обязательные / «будет плюсом» по секциям текста), специализацию и грейд. Технологии сопоставлены навыкам теста (`skills.yaml` : PostgreSQL → `sql` , FastAPI → `python` + `http` , …). **Blueprint** вакансии = 40% ядра

специализации (якоря шкалы и сопоставимость с категорией) + 60% навыков из вакансии пропорционально важности; язык берётся из текста вакансии (Java-вакансия не получает Python «по умолчанию»). Дальше работает тот же CAT. Страница `/test-demo` показывает двух кандидатов одинаковой силы: составы навыков совпадают, формулировки — нет. Blueprint используется в скрининге по вакансии и в регулярных заданиях работодателя.

7. Почему механика устойчива к распространению заданий

1. **Утечка экземпляра бесполезна.** У другого кандидата другие числа, данные и код — и другой ответ. Хранить «ответы к тесту» невозможно: их пространство огромно.
2. **Адаптивность.** Разные кандидаты получают разные семейства: максимальная экспозиция семейства в симуляции — ~40%, медианная — ~8%.
3. **Контроль экспозиции.** Randomesque-выбор и потолок частоты показа.
4. **Знание шаблона ≈ подготовка по теме.** Утечка *решения шаблона* даёт частичный выигрыш (как изучение темы), а не готовый ответ. В симуляции при утечке 50% банка ложные подтверждения грейда: фиксированный тест — **60%**, наш подход — **18%** (см. validation.md, §2.4).
5. **Мониторинг дрейфа (CUSUM).** По каждому семейству копится сумма остатков «факт – ожидание» ($S = \max(0, S + x - p - 0.12)$); при $S > 7$ семейство помечается и выводится из ротации до ревизии. В симуляции утечка, которой пользуется половина кандидатов, ловится во всех случаях медианно за ~75 ответов при ~2% ложных тревог.
6. **Сервер — источник истины.** Правильные ответы и seed не покидают сервер; вариант восстанавливается по HMAC(секрет, попытка, позиция) для апелляций; лимит времени проверяется сервером. Поведенческие сигналы (уход со страницы, копирование, вставка) влияют на результат — см. §9.
7. **Онлайн-перекалибровка.** Параметры семейств уточняются по ответам методом MML-EM (байесовская мода с экспертным приором): ошибка сложности в симуляции снижается с 0.28 до 0.23 по 17 тыс. ответов — «подобравшее» утечку семейство становится видно и по параметрам.

8. Второй этап для Middle, Senior, Lead: практический кейс

Почему. Тест знаний хорошо отделяет Intern/Junior, но грейд выше определяется самостоятельностью, работой с неопределённостью, компромиссами и архитектурой (карьерные матрицы GitLab, Dropbox). По пересмотренному мета-анализу Sackett et al. (2022) самые валидные методы отбора — структурированное интервью ($r = 0.42$), тесты профессиональных знаний (0.40) и рабочие образцы (0.33). При этом короткие вопросы «что выведет программа» легко решаются LLM: в калибровке локальная Qwen3-4B без рассуждений решала ~60% заданий уровней 1–3.

Формат. После теста знаний — кейс уровня грейда (15–18 мин; `ml/fspml/data/scenarios.yaml`: 21 кейс, 7 специализаций × 3 грейда, у каждого параметры варианта) и **уточняющий вопрос по ответу кандидата** с таймером 2 минуты. Вопрос выбирается из заранее написанных: по самому слабому критерию ответа, а если все раскрыты — «смена требований». Неожиданное «почему так?» и изменение условия — то, что сложнее всего пройти пересказом чужого ответа (Meta AI-enabled coding, 2025; conversational exam, Barba & Stegner, 2026).

Сюжет из области кандидата. У кейса один скелет — критерии, веса, вопросы, параметры GRM — и несколько *обёрток* (skins) с разными сюжетами: заголовок, контекст и значения сюжетных переменных `{{var}}`, которые подставляются и в рубрику, и в её ключевые слова. Пример: DS Middle «модель с accuracy 97%» — отток клиентов банка (финтех), детектор фолов по видео матча (CV · спорт), прогноз травм (спорт), классификатор эскалаций в поддержке (NLP), отказ насоса по датчикам (временные ряды · промышленность), повторная госпитализация (медицина). Ловушки те же — дисбаланс классов, утечка будущего, группировка по клиенту/матчу/насосу. Для других специализаций — та же механика: спортивные версии кейсов Middle у backend («медленная история матчей игрока» вместо «списка заказов»), QA (промокод на абонемент фитнес-клуба) и аналитика (возврат билета на матч); кейсы фронтенда, мобильной разработки и DevOps от отрасли почти не зависят. Обёртка выбирается по направлению кандидата (2 балла) и отраслям из опроса (1 балл), у базовой версии свои теги; при повторе кейса берётся ещё не виденная версия. Тест `test_skins_keep_rubric_and_difficulty` проверяет, что эталонный ответ каждой обёртки оценивается так же, как базовый, а слабый — так же слабо.

Соответствие ТЗ. Кейс фиксирован и написан заранее, сложность не генерируется нейросетью. LLM (YandexGPT, контур РФ, без логирования) только **оценивает** ответ по рубрике и не придумывает задание.

Оценка. Рубрика с описанными уровнями (BARS): 0 — не затронуто, 1 — в общих словах, 2 — ожидание грейда, 3 — сверх уровня; по каждому критерию судья приводит цитату из ответа. Ответ кандидата передаётся судье как данные в тегах, инструкции внутри игнорируются (защита от prompt injection). Без LLM работает резервная оценка по ключевым признакам рубрики с пометкой «предварительная». Итог: 70% критерии + 30% уточняющий вопрос → категория 0–3.

Как кейс входит в решение. Категория — политомический ответ в модели градуированных ответов (GRM, Samejima, 1969): $P(k \geq j \mid \theta) = \sigma(a(\theta - b_j))$, $a = 1.8$, $b = \text{cut} - 1.0 / \text{cut} / \text{cut} + 0.9$. Порог «соответствует уровню» совпадает с порогом грейда: кандидат ровно на пороге даёт такой ответ с вероятностью 1/2. Правдоподобие кейса умножается на апостериорное распределение θ после теста знаний, и решение принимается по общей шкале, без ручных весов. Слабый кейс при сильном тесте знаний заметно снижает $P(\theta > \text{порог})$ — именно так отсекается «прошёл квиз с подсказкой». Параметры GRM экспертные и уточняются по пилоту.

9. Надёжность прохождения

`ml/fspml/assessment/integrity.py`. Решение принимается по совокупности сигналов, правило известно кандидату до начала:

- **Уход со страницы** (`visibilitychange` + `blur` окна) дольше 2 с — с длительностью по каждому заданию. При возврате кандидат сразу видит предупреждение и число оставшихся допустимых уходов.
- **Копирование текста задания и вставка в ответ** отключены, попытки считаются.
- **Согласованность ответов** — индекс lz (Drasgow, Levine & Williams, 1985): «верно на трудных, неверно на лёгких». Это мягкий сигнал; у честных симулянтов ложных пометок меньше 6% (тест `test_lz_false_alarm_rate_on_honest_responses`).

Уровень	Условие	Последствие
void	уходы на ≥ 3 заданиях, или суммарно ≥ 60 с вне страницы, или ≥ 3 попытки вставки	попытка аннулируется сразу и считается неудачной (повтор через 14 дней)
review	единичные уходы, копирование/вставка, lz < –2	результат засчитан, работодатель видит замечания
clean	—	«пройден без замечаний»

Ограничение честно: клиентские сигналы не видят второе устройство и оверлеи. Поэтому главная защита — формат: кейс плюс уточняющий вопрос с таймером. Детекторы «текст от нейросети» не используются: они ненадёжны и дают ложные обвинения.

Полноэкранный режим и окружение (все тесты на десктопе, `frontend/src/lib/environment.ts`). Тест открывается в полноэкранном режиме по клику «Начать»; вне его задание скрыто, а время вне режима засчитывается как уход со страницы (уход на вкладку и выход из режима не суммируются — переключение вкладки само выводит из режима). Если браузер отказал в режиме (встроенные окна, политики), тест не блокируется, но работодатель видит пометку. Пометки без аннулирования:

Сигнал	Как определяется	Что ловит / ограничение
Окно уменьшено внутри полноэкранного режима	размер окна < 85% размера при входе в режим	открытые инструменты разработчика
Второй монитор	`screen.isExtended` (Chromium), событие `change`	подсказчик на втором экране; в Firefox/Safari — неизвестно
Удалённый рабочий стол / виртуальная машина	имя видеоадаптера WebGL (`UNMASKED_RENDERER`), классифицирует сервер: «Microsoft Remote Display Adapter», «Basic Render Driver», VMware, VirtualBox, Parallels, Hyper-V, QEMU	сеанс RDP Windows и ВМ; программный рендеринг не считается сигналом (бывает при отключённом ускорении); легальные корпоративные VDI — поэтому только пометка
Автоматизация браузера	`navigator.webdriver`	боты и агенты, управляющие браузером

Ограничение честно: удалённый доступ к физическому компьютеру (TeamViewer, AnyDesk, Chrome Remote Desktop) браузер не видит — для него ввод помощника неотличим от ввода кандидата. Против этого работают проктор (речь, телефон, взгляд, сверка лица) и формат кейса с уточняющим вопросом с таймером. Для production и грейдов Senior/Lead — экзаменационный браузер (Safe Exam Browser: блокирует удалённый доступ, ВМ и демонстрацию экрана; сервер проверяет его по заголовку `X-SafeExamBrowser-ConfigKeyHash`). Перспективный, но требующий пилота сигнал — модель времени ответа (van der Linden): путь «увидеть → спросить ИИ → ввести» даёт долгие паузы на лёгких заданиях с верным ответом.

Водяной знак. На карточке каждого задания — бледный код попытки и кандидата. Экземпляры заданий и так уникальны (`instance_seed`), а код делает источник фотографии очевидным: это сдерживает фотографирование заданий.

9.1 Цифровой проктор (по желанию кандидата)

Закрывает то, чего не видят браузерные сигналы: телефон, которым фотографируют экран; подсказчика рядом; зачитывание заданий вслух; подставного сдающего.

Что работает на устройстве кандидата (`frontend/src/lib/proctor/` , библиотека Human на TF.js, модели раздаются своим nginx из `/models/human` , без внешних CDN):

Детектор	Модель / метод	Счётчик на задание
Лицо в кадре, несколько лиц	BlazeFace	`no_face_ms` , `multi_face_ms`
Телефон в кадре	CenterNet (COCO, класс «cell phone»), эпизод — от 2 кадров подряд	`phone_hits`
Взгляд вбок или вверх	поворот головы по FaceMesh (	yaw
Взгляд вниз (лист, клавиатура)	наклон головы вниз > 29° — разрешён, только счётчик	`look_down_ms`
Речь (зачитывание вслух)	VAD: энергия в полосе 300–3400 Гц выше адаптивного шумового порога на 15 дБ, фраза ≥ 1,5 с	`speech_ms`
Камера закрыта / выключена	трек `ended` / `muted` , кадр тёмный или без контраста	`cam_lost_ms`
Микрофон выключен	трек `ended` / `muted` / `enabled = false` , «цифровая тишина» ≥ 10 с	`mic_lost_ms`
Живое лицо (на проверке перед стартом)	модели liveness и antispoof	— (подсказка кандидату)
Отпечаток лица	faceres → эмбеддинг 1024; только с качественного кадра: одно лицо ≥ 160 px, уверенность детектора ≥ 0.8, лицо смотрит в камеру (	yaw

Видео и звук не записываются и не передаются. На сервер уходят счётчики по заданию, эмбеддинг лица на старте и примерно раз в минуту (в случайный момент) и — при отдельном согласии — один кадр JPEG 320×240 с проверки перед стартом.

Цепочка доверия к личности.

1. *Владелец профиля — реальный человек:* привязка ФСП ID (OIDC) даёт бейдж «Личность подтверждена · ФСП ID»; в production — дополнительно ЕБС.
2. *Все попытки сдаёт один и тот же человек:* первая попытка с проктором создаёт эталон лица, следующие сверяются с ним **на сервере** (`backend/app/modules/biometrics/`). Клиентскому флагу «совпало» не доверяем. Сверка только 1:1 с эталоном своего профиля, поиска по базе (1:N) нет. Сходство считается той же формулой, что `human.match.similarity` (порт сверен с JS на случайных векторах). Эталон — галерея до 5 отпечатков: 3 качественных кадра с первой проверки и лучшие кадры последующих уверенных совпадений (сходство ≥ 0.6; свет, очки, причёска со временем меняются); сходство — максимум по галерее. Решение трёхзначное: ≥ `BIOMETRIC_THRESHOLD` (0.5, как в демо faceid Human) — совпало, < `BIOMETRIC_MISMATCH_THRESHOLD` (0.35) — уверенно другое лицо, между ними — серая зона, которая сигналом не считается. На старте берётся лучший из кадров проверки. Сходство каждой сверки сохраняется в `AttemptItem.proctor` — по нему пороги

калибруются на пилоте. Пересоздание эталона после отзыва согласия видно работодателю: иначе подставной сдающий мог бы незаметно занять место владельца.

3. *Сдававший — тот, кто пришёл на собеседование:* кадр с проверки виден работодателю только после принятого приглашения или отклика (`GET /api/employer/candidates/{id}/test-photo` , запись в журнал доступа).

Правила (`IntegrityConfig` , консервативные: ложное срабатывание детектора не должно аннулировать честного кандидата):

Сигнал	Последствие
Лица нет в кадре дольше 20 с	засчитывается как уход со страницы (общий счётчик: 3 задания — аннулирование)
Телефон в кадре на 1–2 заданиях / на ≥ 3	review / void
Несовпадение с эталоном на старте; эпизод несовпадения во время теста (≥ 2 уверенно плохие сверки подряд)	review
Два эпизода, эпизод вместе с несовпадением на старте, или эпизод из ≥ 3 сверок	void
Один плохой кадр; сверки в серой зоне	не сигнал (поворот головы, наклон к листу, свет)
Несколько лиц; речь на ≥ 2 заданиях; взгляд вбок или вверх на ≥ 3	review
Взгляд вниз (лист с расчётами)	не сигнал: счётчик заданий виден работодателю
Камера или микрофон отключены суммарно ≥ 30 с	не нарушение: бейдж «Тест с проктором» снимается, попытка продолжается

Кандидат видит предупреждения в реальном времени и может исправить ситуацию (убрать телефон, включить микрофон). Локальное предупреждение «лицо не похоже» появляется только после двух уверенно плохих сверок подряд. После перезагрузки страницы тест не продолжается без камеры; время без неё засчитывается как отключение камеры.

Собственное видео — только на проверке перед стартом. Во время теста кандидат видит индикаторы (камера, микрофон, «вы в кадре», телефон), но не картинку: по ней легко найти слепые зоны камеры, например чтобы сфотографировать задание. Если лица нет в кадре дольше 3 с, индикатор подсвечивается.

Расчёты. Ручка и лист разрешены (взгляд вниз не замечание). В тесте есть черновик и калькулятор (`frontend/src/components/scratchpad.tsx` , разбор выражения без `eval`): содержимое не уходит на сервер, вставка извне в черновик считается попыткой вставки, а копирование из черновика в ответ разрешено.

Тяжесть замечаний (`IntegrityReport.severity` , сохраняется в профиле): `hard` — прямые признаки подмены или подсказки: несовпадение лица (старт или эпизод), телефон в кадре, посторонние в кадре на ≥ 2 заданиях, webdriver, удалённый рабочий стол или ВМ, попытки вставки; `soft` — остальные замечания (единичные уходы, копирование, второй монитор, уменьшенное окно, речь, взгляд вбок, lz); `none` — без замечаний.

Ранжирование. Доверие к результату — множитель итогового скора подбора (`TRUST` в `ml/fspml/matching/scoring.py` , подробно — matching.md): проктор без замечаний 1.0, с мягкими 0.9, с тяжёлыми 0.8; без проктора (или проктор прерван) 0.6, с замечаниями 0.5. Результат без наблюдения легко списать, поэтому тест с проктором даже с тяжёлыми замечаниями весит больше чистого теста без проктора; прерванный проктор приравнен к тесту без проктора, иначе выгодно включить проктор и выключить камеру. Кандидат знает об этом заранее (правила теста и окно согласия).

Правовая рамка (требует проверки юристом перед production).

- Эмбеддинг лица, используемый для установления личности, — биометрические персональные данные (ст. 11 152-ФЗ): отдельное письменное согласие `proctoring_biometric` с целью, составом, сроком и правом отзыва. Отказ не закрывает доступ к тесту без проктора.
- Кадр для работодателя — отдельное согласие `proctoring_photo` с явным указанием передачи; срок хранения 180 дней; отзыв удаляет кадры.
- 572-ФЗ: аутентификация по биометрии в production должна идти через ЕБС или аккредитованные системы. Поэтому эталон хранится за интерфейсом `BiometricProvider` : `local_demo` (наша БД, AES-256-GCM) — только для демонстрации; `ebs` — точка интеграции, где эталон хранит ЕБС, а мы проверяем подписанный результат.

Ограничения. Телефон вне поля зрения камеры и наушник детектор не увидит; модифицированный клиент может подделать счётчики (проктор повышает цену обмана, но не делает его невозможным — поэтому сверка лица на сервере, а формат с кейсом и уточняющим вопросом остаётся основной защитой). Порог сходства faceres нужно калибровать на пилоте с реальными веб-камерами: на мелких лицах (< 150 px) сходство разных людей завышено, поэтому отпечаток берётся только с крупного лица. Камера и микрофон доступны браузеру только по HTTPS или на localhost.

10. Ограничения и развитие

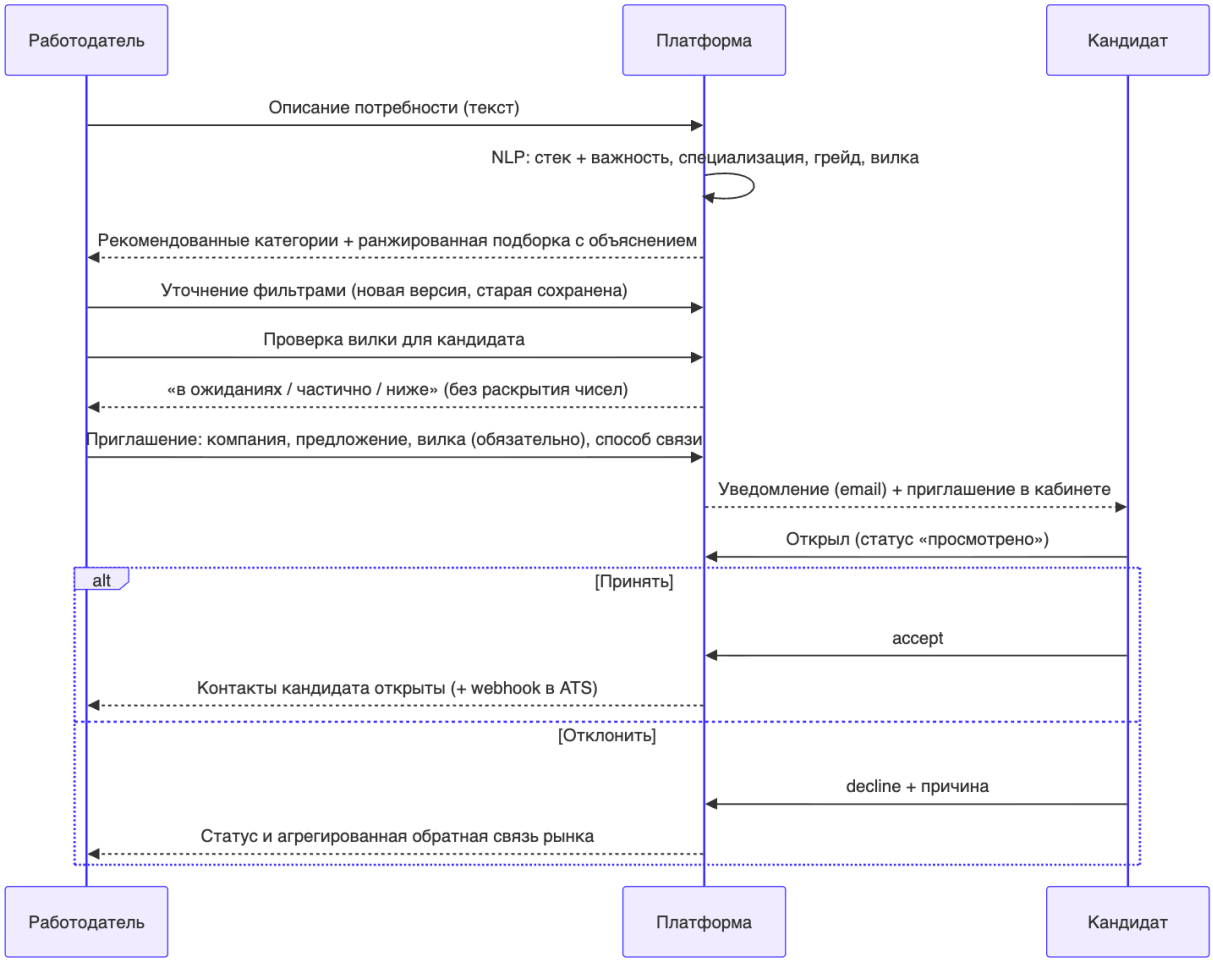
- Начальная калибровка экспертная; нужен живой пилот (протокол — pilot.md) и накопление ответов для MML-EM.
- Совпадение грейда при ретесте упирается в точность измерения на границах грейдов (~75% при 25 заданиях); для продукта это компенсирует правило «грейд не понижается».
- Концептуальные вопросы с выбором имеют небольшое число формулировок; их доля ограничена, приоритет развития — вычислимые форматы.
- Развитие: задачи с исполнением кода кандидата в песочнице как отдельный формат (результат — по тем же правилам IRT), многомерная IRT по навыкам при накоплении данных.

Механика подбора и выхода на контакт

> Код: `ml/fspml/matching/scoring.py`, `ml/fspml/nlp/`, `backend/app/modules/employers/`, `backend/app/modules/invitations/`. Результаты проверки — validation.md.

1. Принцип: инициатива у работодателя

Кандидат не откликается вслепую. Он один раз проходит опрос и тест, получает категорию — и дальше работодатели сами находят его и приходят с конкретным предложением и зарплатой. Классический отклик на вакансию тоже есть, но это дополнительный сценарий.



2. Категоризация

- **Категория** = специализация (опрос) × грейд (тест). Справочник: 7 специализаций × 5 грейдов (`catalog.yaml`).
- В поиск попадают только кандидаты с **подтверждённым** грейдом, согласием на публикацию и статусом поиска «ищу» или «открыт к предложениям».
- Резюме, заявленный стек и самооценка грейда на категорию не влияют: они подсказывают в опросе и используются как «заявленное» при ранжировании.

3. Разбор потребности (NLP, локально)

`parse_need(text, title)`: 1. Секционирование текста: «Требования / Обязательно» (вес 1.0), вступление (0.85), «Задачи» (0.8), «Будет плюсом» (0.45), «Условия» (0.2). Распознаются и строки вида «Будет плюсом: Kubernetes, gRPC», и маркеры внутри одной строки: «Обязательно: Go, PostgreSQL. Будет плюсом: Kafka», «нужен Go-разработчик, обязательно Go, будет плюсом Kafka», «опыт с Kafka будет плюсом»; «не обязательно» требованием не считается. Короткий пункт списка с технологией («- Знание Redis обязательно») — содержимое, а не заголовок. (Раньше однострочное описание целиком уходило в «будет плюсом», а такие пункты терялись: на однострочных вакансиях бенчмарка доля верно размеченных «обязательно / плюс» выросла с 36% до 62%, остальное — рассинхронизация сохранённых текстов с перегенерированной истиной.) 2. Извлечение технологий по словарю `skills.yaml` (~170 технологий с алиасами, русские и английские написания, опечатки — `rapidfuzz`). 3. Специализация — взвешенная сумма индикаторов технологий + сильный сигнал заголовка («Senior Frontend-разработчик»). 4. Грейд — маркер в заголовке → требуемый стаж («опыт от 2 лет») → маркер в тексте. 5. Вилка («250–320 тыс. ₽», «от 300 000 до 400 000 руб», «200-260k ₽») и формат работы. 6. Blueprint теста под вакансию.

Каждый шаг объясняется работодателю («Специализация Backend: заголовок вакансии», «Грейд Middle: требуемый опыт от 2 лет») и может быть исправлен вручную.

4. Ранжирование: два скора

4.1 Сила профиля — порядок *внутри категории*

Используется в банке кандидатов и как компонент выдачи. Ровно по ТЗ: выше те, у кого лучше тест и есть подтверждённые достижения ФСП.

`profile_strength = 0.60·test + 0.25·fsp + 0.15·tasks`

- **test** = Φ((θ – порог грейда) / 0.8) – штраф за неточность (SE > 0.35);
- **fsp** = 1 – Π(1 – s_i) по достижениям (noisy-OR — насыщается: десять участий не равны победе), где s_i = уровень (междунар. 1.0 / всеросс. 0.75 / регион. 0.45 / прочие 0.25) × результат (победитель 1.0 / призёр 0.8 / финалист 0.6 / участник 0.3) × релевантность дисциплины специализации × 0.5^(лет/2);
- **tasks** — регулярные задания с затуханием (полураспад 60 дней).

Нет истории ФСП → fsp = 0, без штрафа; работодатель видит нейтральное «ФСП ID не привязан». Скрытые кандидатом достижения не используются и в ранжировании (иначе косвенная утечка).

4.2 Соответствие потребности — выдача подборки

Версия в продукте — v5 (`rank(..., version=5)`): аддитивная формула v1 ниже, в которой (1) требования вакансии весят по их **специфичности** в категории и (2) жёсткие расхождения работают как **«ворота»** — множители, а не слагаемые. Обоснование и проверка — §4.4.

`match = (0.40·need_fit + 0.30·profile_strength + 0.20·conditions + 0.10·freshness) × ворота` (× 0.85 для соседнего грейда)

- **need_fit** = 0.75·покрытие требуемых технологий + 0.25·семантическая близость текста потребности и профиля (rubert-tiny2 локально или char-n-gram TF-IDF). Подтверждённость технологии t с навыками теста A(t):
- заявлена и навык подтверждён тестом: 0.4 + 0.6·P(θ_навыка > порог грейда потребности) → до **1.0**;
- заявлена, тест по навыку не проводился: ≈ 0.6; заявлена, но тест опроверг: 0.4;
- не заявлена, но навык силён: ≤ 0.25 (общий потенциал);
- технология без навыка в тесте (например, Java): заявлена — 0.6.
- **conditions** = 0.6·вилка + 0.3·формат + 0.1·отрасль — это прокси готовности *принять* приглашение (цель механики — принятые приглашения, а не просмотры).
- **freshness** = exp(–дней с последней активности / 60).
- **Специфичность требования** t в пуле категории: $w_t = \sqrt{(-\ln((df_t + 1)/(N + 2)))}$, где df_t — сколько из N кандидатов заявили t (как IDF в поиске). Покрытие = Σ важность$_t$·w_t·подтверждённость$_t$ / Σ важность$_t$·w_t. Git, Docker, REST есть почти у всех (w ≈ 0.8) и не отличают кандидатов; Go, PHP, Vue, Appium определяют стек вакансии (w ≈ 1.3–1.6).
- **Ворота** (множители 0..1): обязательные требования (важность ≥ 0.6) закрыты по весу меньше чем наполовину — × (0.5 + покрытие обязательного); формат «офис против удалёнки» — × 0.45, через гибрид — × 0.85; ожидания выше вилки больше чем на 15% — линейно до × 0.25. Решение рекрутера устроено как «И»: Java-тимлид без Go не становится релевантным Go-вакансии за счёт сильного теста и Docker с Kafka.
- **Доверие к результату теста** — тоже множитель (`TRUST`). Главное — был ли проктор: результат без наблюдения легко списать, поэтому проктор даже с замечаниями весит больше чистого теста без него. Внутри яруса — по тяжести замечаний попытки, подтвердившей грейд (assessment.md): hard — признаки подмены или подсказки, soft — остальные.

Попытка	×
Проктор, без замечаний	1.00
Проктор, мягкие замечания	0.90
Проктор, тяжёлые замечания	0.80
Без проктора или проктор прерван	0.60
Без проктора, с замечаниями	0.50

При одинаковом соответствии проктор с тяжёлыми замечаниями стоит выше чистого теста без проктора; кандидат без проктора обгоняет, только если подходит в ~1.35 раза лучше (тесты `test_trust_proctor_with_remarks_above_plain` , `test_much_better_fit_without_proctor_still_wins`). На бенчмарках множитель не проверяется: у синтетических кандидатов проктора нет, множитель у всех одинаков и порядок не меняет. - **Опыт работы** — множитель относительно требуемого стажа вакансии (или типичного для грейда): засчитывается релевантный опыт и 30% прочего IT-опыта; нехватка — линейно до × 0.5, превышение — до × 1.06, опыт не указан — × 0.9, от года в отрасли вакансии — × 1.04. - Жёсткие исключения: «не ищу», отрасли/компании, скрытые кандидатом, отозванное согласие, грейд дальше ±1. - **Вакансии и отклики** оцениваются той же формулой: соответствие кандидата вакансии в его ленте и в списке откликов у работодателя считается по v5 со специфичностью требований по пулу специализации, воротами, доверием к тесту и требуемым стажем вакансии («Опыт от, лет»).

4.3 Объяснимость

Каждый скор раскладывается на вклады компонентов (полоска в карточке) и сопровождается причинами:

- ✅ «Подтверждено тестом: Python, PostgreSQL, FastAPI»
- ➖ «Заявлено, но не проверено: Docker, Kafka»
- ✅ «Тест: 82-й перцентиль категории»
- ✅ «ФСП: призёр — всероссийские соревнования, продуктовое программирование, 2025»
- ✅/⚠️ «Ожидания по ЗП в вашей вилке» / «Вилка ниже ожиданий кандидата»

- ⚠️ «Соседняя категория: Senior»
- ❌ «Не подтверждено: Kubernetes» — чего не хватает до полного соответствия; если провалены «ворота» — «Не закрыто из обязательного: Go, gRPC» (самые специфичные пробелы первыми) и «Ожидания по ЗП выше вилки более чем на 15%»

Обоснование весов: компонент соответствия потребности — главный (40%), сила профиля (30%) реализует требование ТЗ о приоритете подтверждённых кандидатов, условия (20%) — практическую применимость. Валидация показывает, что для 7 из 10 весов изменение ±50% меняет NDCG@10 меньше чем на 0.04; наиболее чувствительны штраф соседней категории и вес условий — оба правила содержательно обоснованы.

4.4 Почему v5: разбор ошибок v1 и проверка

Разбор ошибок. На реалистичном бенчмарке (bench2, 700 кандидатов с живыми текстами резюме, 30 вакансий, слепая разметка) у v1 в топ-10 было 148 из 300 мест с оценкой асессора 0–1. Главная причина (~60% случаев) — **несовпадение основной технологии**: на «Senior Go» — Java-, Python- и Kotlin-лиды, на PHP/Laravel — junior на Java, C#, Go и Node.js, на Vue — React- и Angular-разработчики, на iOS — Android, на Java+Selenium+Appium — автоматизаторы на Python и ручные тестировщики, на CV — табличный DS. Механизм простой: в покрытии все требования весили одинаково, и Java-разработчик с Docker, Git, PostgreSQL, Kafka закрывал PHP- или Go-вакансию почти так же, как профильный (покрытие 0.41 против 0.43), а сила профиля и условия решали остальное. Вторая причина — несовместимые условия (формат, вилка), которые аддитивная формула не отсекает.

Проверка (nDCG@10, condensed; Δ и p — к v1 с тем же разбором вакансий, парный рандомизационный тест; варианты заданы до прогона, коэффициенты не подбирались: «ворота» — из v2, специфичность — стандартный IDF):

	bench2, Claude (слепо)	bench2, скрытая истина	bench1, GPT-5.6 Terra	bench1, Claude	bench1, скрытая истина
v1	0.704	0.767	0.789	0.801	0.690
v5	**0.792** (+0.088, p < 0.001)	0.729 (–0.038)	**0.803** (+0.013, p = 0.07)	**0.843** (+0.042, p = 0.002)	0.700 (+0.010)
v5 ⊕ кросс-энкодер	**0.813** (+0.108, p < 0.001)	0.750 (–0.017)	0.796 (+0.006)	0.813 (+0.012)	0.685 (–0.005)
для сравнения: v1 ⊕ кросс-энкодер (прежний гибрид)	0.736 (+0.032)	0.767	—	—	—

v1 здесь — с исправленным разбором вакансий (на однострочных вакансиях bench1 это само по себе +0.010…+0.011). Выигрыш устойчив к форме веса специфичности (ln, √, с потолком, 0.5 + 0.5·ln: +0.08…+0.11 на bench2 и +0.013…+0.019 / +0.031…+0.042 на bench1); выбран √ — мягкий вариант, меньше других расходящийся со скрытой истиной. Кросс-энкодер добавляет пользу только на живых текстах (bench2), поэтому в продукте по умолчанию — формула v5, гибрид включается `FSP_RANKER=hybrid`. Скрытая истина bench2 с v5 расходится — и расходится там, где сама неправдоподобна: она ставит «3» Java-тимлиду без Go на вакансию Senior Go и «0» автоматизаторам Java+Selenium+Appium на вакансию QA Automation (Java), потому что генератор истины не моделирует важность основного языка. Подробности, абляции и ограничения — bench.md.

5. Рекомендованные категории

Для потребности показываются целевая категория, соседние грейды и смежные специализации (если разбор дал им ≥ 15%): число кандидатов, сколько с достижениями ФСП, медиана ожиданий, доля «в вашей вилке». Это реализует сценарий LHH: работодатель сначала видит структуру рынка, затем — людей.

6. Уточнение без потери подборки

Подборка — сохраняемая сущность (`searches`). Фильтры (грейд, обязательный стек, подтверждённые тестом навыки, наличие ФСП, формат, «вилка не ниже ожиданий», «активно ищущие», без соседних грейдов) применяются поверх исходной выдачи; каждое изменение сохраняется версией (`search_versions`), исходная выдача не теряется — в интерфейсе уточнения сбрасываются одной кнопкой; показывается «N из M». Кандидатов можно закрепить в шорт-листе. Подборка пересчитывается при открытии — новые кандидаты появляются автоматически.

7. Выход на контакт

Правило ТЗ	Реализация
Зарплата обязательна («от–до», ₽) — и в вакансии, и в приглашении	Валидация API (422) + CHECK-ограничение в БД
Приглашение: описание, вилка, компания, способ связи	Поля `invitations`
Контакты — только после принятия или отклика	`invitations/access.py`; карточки и PDF обезличены (псевдоним)
Статусы видят обе стороны	sent → viewed (кандидат открыл) → accepted / declined (с причиной) / withdrawn

Дополнительно: - **индикатор вилки** до отправки — без раскрытия ожиданий кандидата; - **квота** приглашений на компанию в неделю (по умолчанию 30) и повторное приглашение того же кандидата запрещено, пока активно предыдущее; - кандидат видит **статистику компании**: сколько приглашений за 90 дней, какая доля принимается, отвечает ли на отклики — основа репутации и анти-фрода; - **обратная связь рынка** работодателю: доли причин отказов, медиана и квартили ожиданий в категориях, где он ищет, доля отказов по зарплате; - **webhook в ATS** при принятии приглашения.

8. Классический отклик (дополнительно)

Лента вакансий кандидата отсортирована той же формулой «в обратную сторону» (кандидат относительно вакансии) и показывает, почему вакансия подходит. Отклик сразу открывает контакты работодателю; отклики работодатель видит отсортированными по соответствию, со статусами viewed → interview → rejected / hired.

Валидация решения

> Сгенерировано `make validate` (377.0 с). Все числа воспроизводимы: генераторы детерминированы по seed. Сырые результаты — `docs/validation.json`.

1. Постановка

Готовых данных и эталонной разметки в задаче нет, поэтому валидация построена на **синтетической популяции со скрытой истиной** (1000 кандидатов, 30 потребностей работодателей) плюс проверки на реальных текстах вакансий. Система никогда не видит истинных признаков: только ответы в тесте, заявленный стек, текст резюме, достижения ФСП и условия.

Чтобы валидация не была цикличной, ответы на задания симулируются по модели, *отличной* от модели оценивания:

- истинная сложность семейства отклоняется от номинальной (σ = 0.35), дискриминативность — в 1.2 раза в обе стороны;
- экземпляры одного семейства неидеально равны по сложности (σ = 0.25);
- кандидат отвечает с навыко-специфичной способностью (σ = 0.45 от общей) — тест многомерен, модель одномерна;
- угадывание в заданиях с выбором реально происходит.

Эталонная релевантность пары «потребность–кандидат» (0–3) задана по скрытым признакам *до* формулы ранжирования: реальное владение требуемыми технологиями, истинная сила относительно порога грейда (знания + практика), готовность к предложению (статус поиска, вилка, формат).

Допущения генератора (их стоит проверить на живом пилоте): 30% кандидатов «накручивают» стек, чаще — более слабые и в тех областях, где они слабее; достижения ФСП отражают практический навык, частично независимый от знаний, которые меряет тест; у ~60% кандидатов истории ФСП нет.

2. Механика тестирования

2.1 Банк заданий

- **303 параметризованных семейств** по 31 навыкам, уровни сложности 1–5;
- 65.7% семейств — с вводом ответа (угадывание ≈ 0, утёкший ключ не подходит к новому экземпляру);
- из 200 генераций семейства медианно **174 уникальных экземпляров**; ≥100 уникальных — у 71.3% семейств, <10 — у 3.3% (концептуальные вопросы с выбором; их доля в тесте ограничена blueprint).
- Каждое семейство проходит автопроверки в CI: правильный ответ вычисляется исполнением кода/SQL и принимается проверкой, варианты не дублируются, ответ меняется между экземплярами; вывод заданий на JavaScript, Java, Go, C# и PHP сверяется с настоящими рантаймами.

2.2 Точность и стабильность грейда

Онбординг: тест на заявленный грейд → при неудаче на ступень ниже → при уверенном успехе одна попытка на ступень выше. Повтор (ретест) — второй онбординг того же кандидата; как и в продукте, система помнит семейства прошлых попыток и выбирает другие, то есть ретест идёт на *параллельной форме*, а не на тех же шаблонах с другими числами.

Метрика	Значение
Совпадение с истинным грейдом	**67.8%** (±1 ступень — 99.4%)
Завышение / занижение грейда	2.7% / 29.5%
Cohen's κ с истиной	0.56
Ретест: совпадение грейда	**69.3%**
Ретест: κ / взвешенная κ	0.57 / **0.82**
Ретест: корреляция θ	0.82
Ретест: совпадение решения по заявленному грейду	77.2%
Корреляция оценки θ с истинной	0.89
Заданий за попытку / за онбординг	19.0 / 30.9
Повтор по сути (то же семейство или группа): в следующей попытке онбординга / на ретесте	12.7% / 28.2%
Точность вне пограничной зоны (	θ – порог



Интерпретация. Ошибки — только на соседний грейд и в основном *в сторону занижения*: для работодателя подтверждённый грейд надёжен (завышение 2.7%), а кандидат может пройти тест на ступень выше после кулдауна. Совпадение при ретесте ограничено точностью измерения: при SE ≈ 0.33 и ширине грейда ≈ 1.0 по шкале θ теоретический потолок совпадения для кандидатов на границе грейдов — около 70%. Взвешенная κ, учитывающая порядковость грейдов, — в зоне «почти полного согласия». Для продукта важнее, что **грейд не меняется вниз при повторе по правилам платформы**: повторная попытка может только подтвердить грейд или повысить его.

Выбор порога решения (P(θ > порог) для подтверждения):

Порог	Точность	Завышение	Занижение	Совпадение ретеста	Заданий
0.5	71.2%	3.4%	25.4%	70.2%	29.8
0.6	67.6%	2.0%	30.4%	69.4%	31.8
0.7	63.2%	2.0%	34.8%	70.6%	33.1
0.8	55.8%	1.2%	43.0%	69.0%	35.2

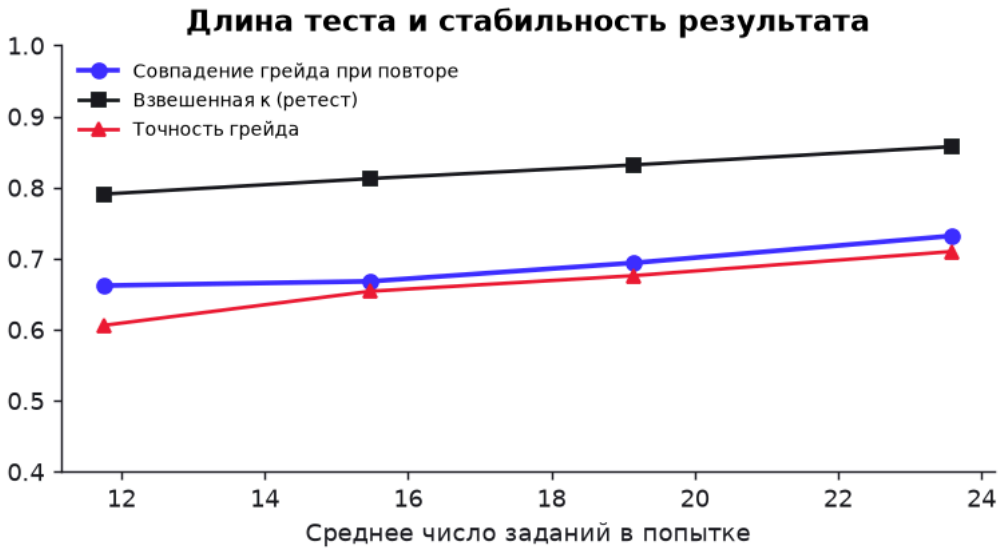
Отдельно — одна попытка теста: доля ложных подтверждений, когда кандидат заявляет грейд на ступень выше истинного, против доли подтверждений при честной заявке:

Порог	Ложные подтверждения (+1 ступень)	Верные подтверждения
0.50	9.8%	76.4%
0.60	6.8%	69.8%
0.70	3.6%	61.2%
0.75	3.2%	56.4%
0.80	2.8%	52.8%

Выбрано: первичное подтверждение при **0.6** — компромисс: точность итогового грейда близка к максимальной, завышение не выше 5%, а ложных подтверждений при завышенной заявке меньше, чем при 0.5 (6.8% против 9.8%). Неудача не наказывается — кандидат сразу проходит тест на ступень ниже. **Повышение** уже подтверждённого грейда — при **0.75** (ложные подтверждения 3.2%): ложное повышение дороже. Предложение пройти тест выше — при P(θ > порог следующего) ≥ 0.8.

Длина теста. Стабильность растёт с длиной теста; 25 заданий (~20–30 минут) — компромисс между точностью и временем кандидата.

Макс. заданий	Заданий в попытке	Совпадение ретеста	Взвеш. κ	Точность	r(θ̂, θ)
12	11.8	66.2%	0.79	60.6%	0.86
18	15.5	66.8%	0.81	65.4%	0.89
25	19.2	69.4%	0.83	67.6%	0.89
35	23.6	73.2%	0.86	71.0%	0.90



Память о показанных заданиях. Без неё следующая попытка (например, Middle после Junior) на 52.0% состоит из уже виденных шаблонов, ретест — на 60.8%: кандидат узнаёт задачи, отличаются только числа. С памятью — 12.7% и 28.2%; остаток — навыки, где семейств меньше, чем заданий за две попытки (тогда берутся давно виденные, тест не обрывается). Точность грейда — 67.8% против 70.0% без памяти. Совпадение ретеста (69.3% против 72.0%, взвешенная к 0.82 против 0.86): повтор на тех же шаблонах завышал стабильность за счёт общей для шаблона ошибки измерения, параллельная форма — более строгая и честная оценка.

Проверенная и отклонённая гипотеза: учитывать ответы предыдущих попыток онбординга в следующей (продолжать измерение). Это сокращает тест до 25.6 заданий, но не повышает точность (70.0%) и стабильность (69.6%); по умолчанию выключено.

2.3 Состав теста: ядро специализации + модули стека

Раньше состав теста был общим для специализации, а стек выбирал только язык (при отсутствии совпадения подставлялся Python): Java-, Go-, C#-разработчики получали пятую часть теста по чужому языку, Vue-разработчики — задания по React, Linux-администраторы — по Kubernetes. Теперь тест = **ядро специализации** (общее для всех в категории — сопоставимость грейдов) + **до трёх модулей по заявленному стеку** (доля модулей ≤ 40%); технологии без заданий ничем не подменяются. Каждый навык состава получает не меньше 2 заданий, чтобы его подоценка не оставалась «нет данных».

Эксперимент: 600 кандидатов с реалистичными стеками (архетипы рынка), backend-разработчик знает только свой язык. У 83.2% из них прежний состав содержит навыки не из реального стека. Штраф за чужой стек — насколько способность в навыке-модуле не из реального стека ниже общего уровня: 0 — стек не влияет на ответы (как в остальной валидации), 0.5–1.0 — реалистичный диапазон.

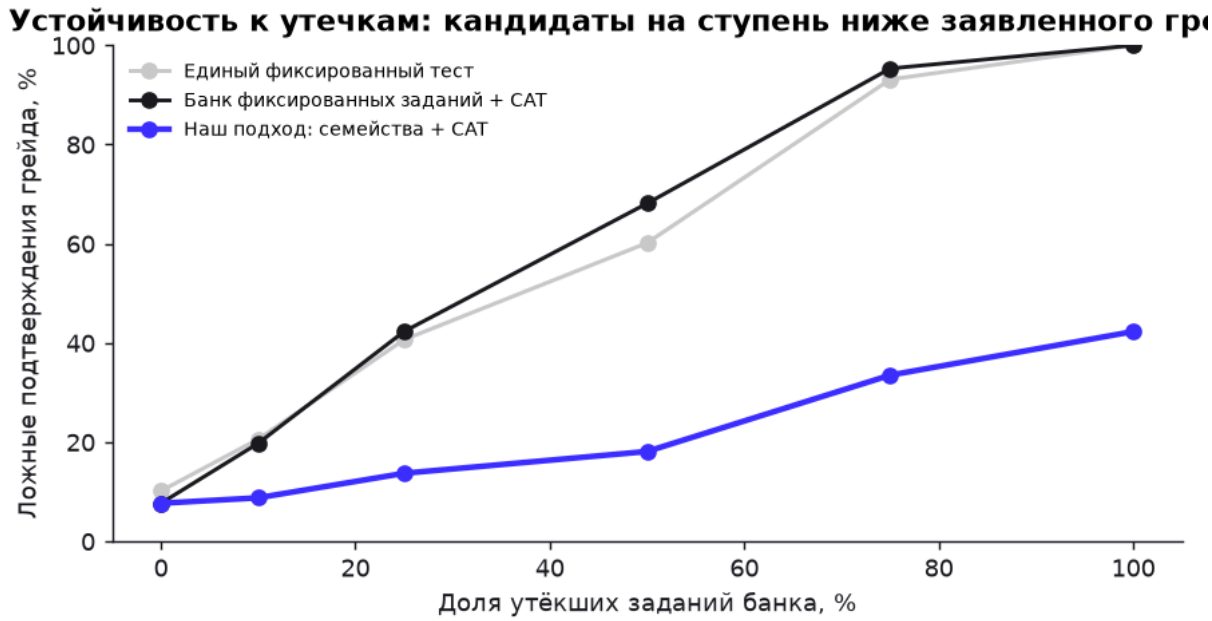
Штраф за чужой стек	Состав	Точность грейда	Точность у затронутых	Занижение у затронутых	Смещение θ у затронутых	Ретест	Заданий	Заявленных технологий с данными теста	Подтверждено: накрученные / реальные
0.0	прежний общий	65.2%	64.9%	33.3%	-0.10	70.5%	17.9	74.6%	75.9% / 76.9%
0.0	**ядро + модули**	64.5%	64.5%	32.7%	-0.11	70.2%	19.0	92.5%	77.5% / 78.6%
0.5	прежний общий	60.3%	59.7%	38.3%	-0.18	68.8%	18.3	74.9%	70.2% / 78.1%
0.5	**ядро + модули**	62.3%	61.5%	36.9%	-0.15	70.7%	18.9	92.5%	72.5% / 78.7%
1.0	прежний общий	55.0%	53.5%	44.9%	-0.25	70.7%	18.5	76.0%	68.8% / 76.1%
1.0	**ядро + модули**	63.7%	62.9%	34.5%	-0.13	73.3%	18.9	92.5%	76.7% / 81.0%

Итог. Когда стек не влияет на ответы, новый состав сопоставим с прежним (точность 64.5% против 65.2% — в пределах шума, ретест 70.2% против 70.5%). Когда влияет (штраф 1.0), прежний состав систематически занижает оценку затронутых кандидатов (смещение θ -0.25, занижение грейда 44.9%), новый — нет (-0.13, 34.5%). Доля заявленных технологий, по которым у работодателя есть данные теста, растёт с 76.0% до 92.5% — это прямой вход механики подбора («подтверждено тестом», §4), и накрутка стека чаще упирается в проверку. Цена — +0.4 задания на попытку.

2.4 Устойчивость к утечкам

Сценарий: 364 кандидатов пытаются подтвердить грейд на ступень выше истинного и имеют доступ к утечкам. Для фиксированных заданий утёкший ответ решает задание ($p = 0.98$). В нашем подходе утечка экземпляра бесполезна (другие параметры — другой ответ), а знакомство с шаблоном даёт лишь частичный выигрыш (как от подготовки по теме).

Доля утёкшего банка	Единый тест	Банк фикс. заданий + CAT	Наш подход
0%	10.2%	7.7%	**7.7%**
10%	20.6%	19.8%	**8.8%**
25%	40.7%	42.3%	**13.7%**
50%	60.2%	68.1%	**18.1%**
75%	93.1%	95.3%	**33.5%**
100%	100.0%	100.0%	**42.3%**



Мониторинг утечек (CUSUM). На 60 «честных» семействах по 600 ответов ложных тревог — 1.7%. Если решение шаблона утекло и им пользуется 50.0% кандидатов, семейство помечается в 100.0% случаев, медианно через 75 ответов (90-й перцентиль — 158), и выводится из ротации.

2.5 Дискриминативность и калибровка

- Использовано семейств в симуляции: 298 из 303; оценено (≥30 ответов): 207.
- Медианная точечно-бисериальная корреляция «верно ↔ истинная способность»: **0.36**; доля семейств с r < 0.2 — 8.2% (кандидаты на доработку: alg.knapsack, docker.layers, http.rest_design, java.stream_lazy, js.prototype, js.typeof, ml.kmeans, ml.knn).
- Экспозиция: максимум 41.4% кандидатов видели одно семейство, медиана — 4.7%.
- **Онлайн-перекалибровка (MML-EM)** по 19047 ответам: средняя ошибка сложности b для 207 семейств снижается с 0.27 (экспертная разметка) до **0.22**. То есть по мере работы платформы параметры банка уточняются по данным.

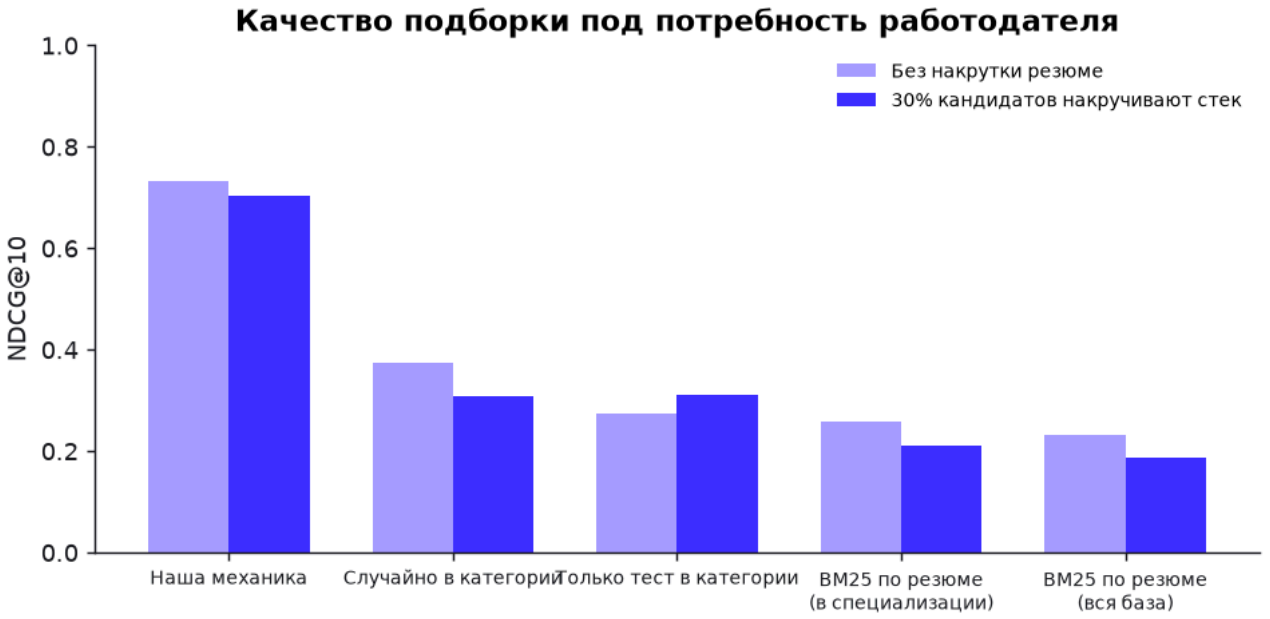
3. Категоризация

Категория = специализация (из опроса) × подтверждённый тестом грейд. Категоризированы 99.3% кандидатов; грейд категории совпадает с истинным у **67.6%**, ±1 ступень — 99.5%. Резюме на категорию не влияет.

4. Механика подбора

Тестовый набор: 30 потребностей × 1000 кандидатов, эталон — релевантность 0–3 по скрытым признакам. Метрики на топ-10: NDCG@10 (градуированная), P@10 (доля релевантных ≥ 2), MRR, ожидаемое число *принятых* приглашений среди релевантных (по модели решения кандидата).

Система	NDCG@10	P@10	MRR	Принятые@10	NDCG@10 без накрутки
Наша механика	0.703	0.647	0.917	3.77	0.734
— без ФСП	0.695	0.643	0.883	3.79	0.735
— без подоценок навыков из теста (только заявленный стек)	0.669	0.633	0.889	3.72	0.739
— без совместимости условий	0.404	0.337	0.587	1.67	0.420
Исходная формула v1 (до специфичности и «ворот»)	0.613	0.560	0.925	3.11	0.657
Только результат теста внутри категории	0.311	0.307	0.486	1.63	0.275
Случайный порядок внутри категории	0.308	0.297	0.472	1.53	0.374
BM25 по тексту резюме в специализации	0.211	0.213	0.386	1.17	0.257
BM25 по тексту резюме, вся база («универсальная площадка»)	0.188	0.190	0.363	1.01	0.233



Итог. Механика даёт NDCG@10 = 0.70 против 0.19 у поиска по ключевым словам на «универсальной площадке» и в 3.7 раза больше ожидаемых принятых приглашений в топ-10. При накрутке стека у 30% кандидатов качество BM25 падает на 18%, нашей механики — на 4%.

Опыт работы. У синтетических кандидатов есть история работы: часть джунов без коммерческого опыта, около 20% сменили роль (общий стаж больше релевантного); в потребностях указан требуемый опыт. С учётом релевантного опыта NDCG@10 = 0.703, без него (история скрыта) — 0.631; ожидаемые принятые приглашения в топ-10: 3.77 против 3.50. Оговорка: эталонная релевантность сама понижает кандидатов с нехваткой опыта (так рассуждает работодатель), поэтому прирост показывает согласованность механики с этим правилом, а не независимое подтверждение веса.

v5 против исходной формулы v1 на этой синтетической истине: NDCG@10 0.703 против 0.613, P@10 0.647 против 0.560. Истина генератора считает долю известных технологий вакансии без учёта того, какие из них определяют стек (Go в Go-вакансии весит как Git), — ровно то, что исправляет v5, поэтому здесь v5 не выигрывает. По независимым слепым судьям на реалистичных текстах v5 значимо лучше v1 — bench.md.

Что вносит вклад (абляции). Основной прирост дают категоризация по тесту и совместимость условий (без неё NDCG падает сильнее всего). Подоценки навыков из теста дают +0.034 NDCG@10: состав теста строится по заявленному стеку, поэтому заявленные технологии проверяются тестом (при прежнем общем составе подоценки подбору не помогали — см. разложение ниже). ФСП на синтетике даёт малый прирост: он коррелирует с уже измеренной способностью; его ценность — в доверии работодателя к объяснению, вес ограничен (≈7.5% итогового скора), чтобы он не перекрывал соответствие потребности.

Состав теста × формула подбора (NDCG@10; в скобках — без подоценок из теста, только заявленный стек):

Состав теста	Формула v1	Формула v5
прежний общий	0.632 (0.635)	0.672 (0.685)
ядро + модули стека	0.613 (0.622)	0.703 (0.669)

Прежняя связка (общий состав + v1) — 0.632, и подоценки из теста ей мешали (-0.002): проверялись навыки не из стека кандидата. Новая (ядро + модули + v5) — 0.703, подоценки дают +0.034.

Чувствительность к весам (изменение NDCG@10 при умножении веса на 0.5 / 1.5):

Вес	×0.5	×1.5
`need_fit`	+0.004	-0.019
`profile`	-0.007	-0.012
`conditions`	-0.033	+0.007
`freshness`	-0.000	-0.003
`ps_test`	-0.004	-0.006
`ps_fsp`	-0.006	-0.006
`coverage_share`	-0.004	-0.016
`claim_base`	+0.001	+0.001
`unclaimed_factor`	-0.014	-0.002
`adjacent_grade_multiplier`	-0.014	-0.017

Базовое NDCG@10 = 0.703. Для 10 из 10 весов изменение в пределах ±50% меняет NDCG меньше чем на 0.04 — формула не «подогнана» под синтетику. Наибольшая чувствительность — к `conditions` (до 0.033): это ожидаемо и подтверждает обоснованность соответствующего правила.

5. Разбор текстов (NLP)

Вакансии. На 7 демонстрационных вакансиях (`ml/fspml/data/vacancies.yaml`) специализация определена верно в 100.0%, грейд — в 100.0%. Blueprint теста под вакансию:

Вакансия	Blueprint (доли навыков в тесте)
fintech-python-middle	SQL и базы данных 23%, Архитектура и проектирование 22%, HTTP, REST и сети 21%, Python 12%, Алгоритмы и структуры данных 11%, Docker 5%
ecom-react-senior	HTML и CSS 36%, JavaScript 30%, React 19%, HTTP, REST и сети 9%, Алгоритмы и структуры данных 6%
telecom-ds-junior	Python 30%, Машинное обучение 28%, Статистика и вероятность 23%, SQL и базы данных 12%, pandas / NumPy 7%
gov-qa-middle	Тест-дизайн и теория тестирования 57%, HTTP, REST и сети 23%, Python 8%, Git 6%, SQL и базы данных 6%
industry-devops-senior	Linux и shell 29%, CI/CD 23%, Docker 15%, HTTP, REST и сети 13%, Архитектура и проектирование 9%, Kubernetes 6%
bank-analyst-middle	Требования и моделирование 50%, SQL и базы данных 27%, HTTP, REST и сети 13%, Продуктовые метрики 6%, Архитектура и проектирование 5%
edtech-android-middle	Мобильные платформы 59%, HTTP, REST и сети 18%, Git 10%, Алгоритмы и структуры данных 6%, Архитектура и проектирование 6%

Резюме. 60 синтетических PDF (разные заголовки секций, форматы дат, опечатки в названиях технологий) → парсер: точность полей name 100.0%, email 100.0%, phone 100.0%, telegram 100.0%, city 100.0%, specialization 100.0%; технологии P/R/F1 = 0.97/1.00/0.99; стаж ±0.5 года — 100.0%. Синтетические PDF чище реальных — следующий шаг: 10–15 реальных обезличенных резюме с ручной разметкой.

6. Ограничения и следующие шаги

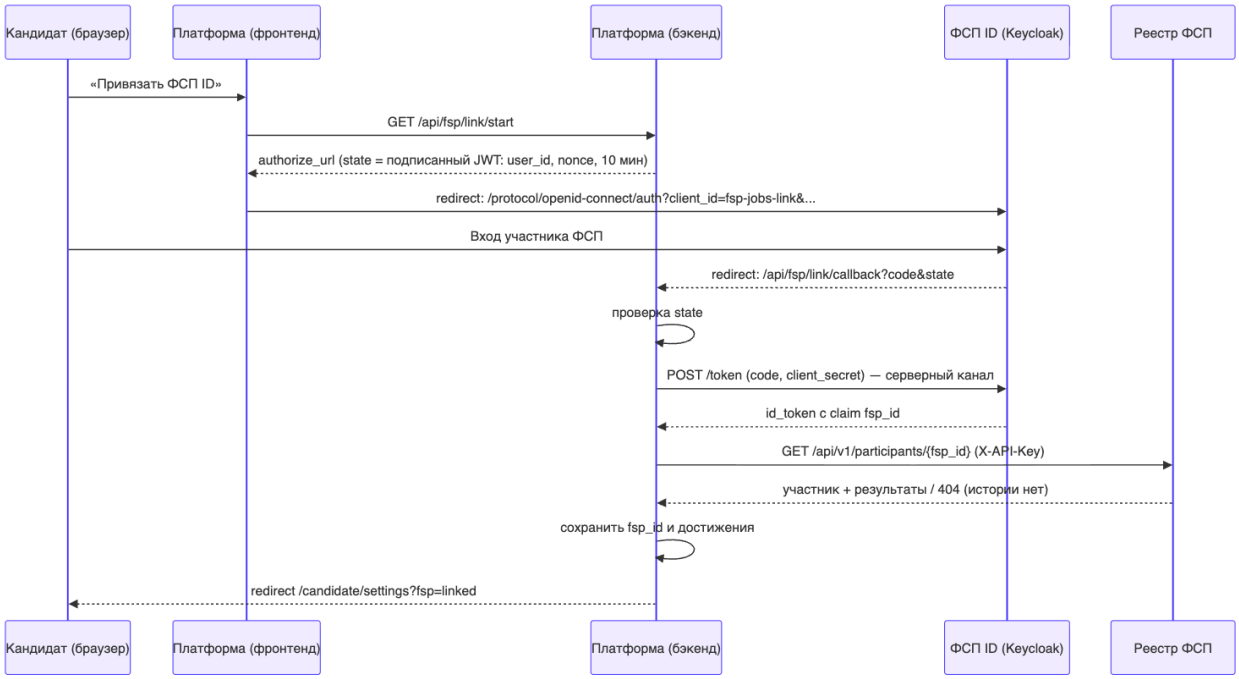
- Параметры семейств — экспертные; после запуска — онлайн-перекалибровка (показана выше), мониторинг дрейфа и вывод слабых семейств.
- Живой пилот (15–20 человек): сравнение грейда с экспертной оценкой команды и интервьюеров — протокол в `docs/pilot.md` .
- Эталонная релевантность синтетическая; на реальных данных — обучение весов по откликам на приглашения (learning to rank) с контролем объяснимости.

Схема интеграции с ФСП ID и реестром ФСП

ТЗ: API и структура данных ФСП не предоставляются — команда сама решает, какие сведения получать и как показывать; обязательна корректная обработка отсутствия истории. Ниже — предлагаемая схема. В демо она работает на имитациях: realm `fsp-id` в Keycloak (ФСП ID) и сервис `fsp-registry-mock` (реестр).

1. Привязка профиля к ФСП ID

ФСП ID будет работать на Keycloak, поэтому привязка — стандартный OAuth 2.0 / OpenID Connect *authorization code flow*. Платформа выступает OIDC-клиентом `fsp-jobs-link` (конфиденциальный, с секретом).



Переход на реальный ФСП ID — смена `KEYCLOAK_PUBLIC_URL` / `FSP_ID_REALM` / `FSP_ID_CLIENT_ID` / `FSP_ID_CLIENT_SECRET` и маппинга claim `fsp_id`. Один ФСП ID привязывается только к одному профилю (409 при попытке повторной привязки). Отвязка удаляет идентификатор и достижения.

2. Предлагаемые данные реестра

Сущность	Поля	Зачем платформе
Участник	`fsp_id`, ФИО, регион, спортивный разряд/звание, `updated_at`	Сверка личности, разряд — дополнительный сигнал
Соревнование	`id`, название, дисциплина (`algorithmic`, `product`, `security`, `robotics`, `uav`), уровень (`international`, `national`, `regional`, `local`), даты, город, организатор	Вес достижения и релевантность специализации
Результат	`id`, `event_id`, статус (`winner`, `prize`, `finalist`, `participant`), место, дата, роль в команде, команда	Сила достижения, роль (backend/frontend/капитан…) для работодателя

Формат ответа `GET /api/v1/participants/{fsp_id}`:

```
{
  "fsp_id": "FSP-000123",
  "full_name": "Иванова Мария",
  "region": "Москва",
  "sport_rank": "КМС",
  "updated_at": "2026-09-30T00:00:00",
  "achievements": [
    {"id": "FSP-000123-R1", "event_id": "EV-0007",
     "event": "Всероссийские соревнования по продуктовому программированию 2025",
     "discipline": "product", "level": "national", "result": "prize", "place": 2,
     "date": "2025-11-20", "role": "backend", "team": "Команда «Сигма»"}
  ]
}
```

- `404` — участника нет в реестре: **история ФСП отсутствует**, профиль полноценный.
- Синхронизация: при привязке, по кнопке «Обновить» и фоновой задачей (раз в `FSP_SYNC_INTERVAL_HOURS`, по умолчанию 6 ч); для масштаба — дельта `GET /api/v1/participants?changed_since=…`.
- Аутентификация сервис-сервис — API-ключ (в продакшене — mTLS или client credentials того же Keycloak).
- Справочник соревнований: `GET /api/v1/events`.

3. Как достижения используются и показываются

- **Ранжирование:** компонент `fsp` силы профиля (25%) — noisy-OR по достижениям с учётом уровня, результата, релевантности дисциплины специализации и давности (полураспад 2 года). Матрица релевантности дисциплин — `catalog.yaml` (например, продуктовое программирование → Backend/Frontend/Mobile 1.0; ИБ → DevOps 1.0).
- **Карточка кандидата:** «ФСП: призёр — всероссийские соревнования, продуктовое программирование, 2025 (+1)», список достижений с ролью в команде, фильтр «есть подтверждённые достижения ФСП», число кандидатов с ФСП в каждой категории.
- **Отсутствие истории** — нейтрально: «ФСП ID не привязан» или «ФСП ID привязан, достижений пока нет»; без штрафа к профилю.
- **Приватность:** кандидат может скрыть достижения; скрытые не участвуют и в ранжировании.

4. Вопросы к ФСП для реальной интеграции

См. questions-fsp.md: какие данные реально есть в реестре, гарантии актуальности, формат ФСП ID (claim с идентификатором участника), доступ сервис-сервис, юридическая база передачи данных (согласие участника на передачу в платформу — получается в момент привязки).

Описание API

> Сгенерировано из OpenAPI-схемы (`docs/tools/gen_docs.py`). Интерактивная документация — **Swagger UI** `/api/docs` , ReDoc — `/api/redoc` , схема — `/api/openapi.json` и openapi.json.

Общие соглашения

- **Аутентификация:** `Authorization: Bearer <access_token>` — токен OIDC из Keycloak (realm `fsp-jobs` , клиент `fsp-jobs-web` , PKCE). Бэкенд проверяет подпись по JWKS, срок, issuer и audience `fsp-jobs-api` . Публичные методы: справочники, конфигурация авторизации, демо «тест из вакансии».
- **Роли:** кандидат / работодатель — выбираются при первом входе (`POST /api/me/role`). Чужие объекты возвращают `404` (не раскрываем факт существования), неверная роль — `403` .
- **Ошибки:** единый формат `{"error": {"code": "...", "message": "...", "details": ...}}` .

HTTP	Пример code	Когда
400	`consent_required` , `specialization_required` , `pdf_unreadable`	Некорректный запрос
401	`unauthorized` , `invalid_token`	Нет или недействителен токен
403	`wrong_role` , `email_not_verified`	Недостаточно прав
404	`not_found`	Объект не найден или чужой
409	`attempt_not_allowed` , `already_invited` , `survey_required` , `fsp_id_taken`	Конфликт состояния (кулдаун, повтор, нет предусловия)
422	`validation_error`	Ошибка валидации полей (например, вилка «до» меньше «от»)
429	`quota_exceeded`	Квота приглашений исчерпана
503	`registry_unavailable` , `idp_unavailable`	Недоступен внешний сервис

Пользователь

Профиль пользователя, выбор роли, согласия на обработку ПДн (152-ФЗ).

Метод	Путь	Назначение	Коды ответов
`GET`	`/api/auth/config`	Параметры авторизации для фронтенда	200
`GET`	`/api/me`	Текущий пользователь и шаг онбординга	200, 401, 422
`GET`	`/api/me/consents`	История согласий	200, 401, 422
`POST`	`/api/me/consents`	Дать или отозвать согласие	200, 401, 422
`POST`	`/api/me/role`	Выбор роли при первом входе	200, 400, 401, 409, 422

Справочники

Специализации, грейды, навыки теста, отрасли, ФСП.

Метод	Путь	Назначение	Коды ответов
`GET`	`/api/catalog`	Справочники: специализации, грейды, навыки, отрасли, ФСП	200
`GET`	`/api/catalog/demo-vacancies`	Демонстрационные описания вакансий (для показа генерации тестов)	200
`GET`	`/api/catalog/validation`	Сводка результатов валидации механик (генерируется make validate)	200
`GET`	`/api/health`	Проверка работоспособности	200

Кандидат: профиль

Профиль, обязательный опрос, разбор PDF-резюме, PDF-профиль, приватность.

Метод	Путь	Назначение	Коды ответов
`GET`	`/api/candidate/dashboard`	Сводка кабинета кандидата	200, 401, 403, 422
`PUT`	`/api/candidate/privacy`	Настройки приватности	200, 401, 403, 422
`GET`	`/api/candidate/profile`	Мой профиль	200, 401, 403, 422
`PUT`	`/api/candidate/profile`	Заполнить профиль вручную	200, 401, 403, 422
`GET`	`/api/candidate/profile.pdf`	Стандартизированный PDF-профиль	200, 401, 403, 422
`POST`	`/api/candidate/resume`	Загрузить PDF-резюме и распознать поля	200, 400, 401, 403, 422
`PUT`	`/api/candidate/survey`	Пройти/обновить обязательный опрос	200, 401, 403, 422

Тестирование

Адаптивный тест на заявленный грейд (IRT/CAT, параметризованные семейства заданий).

Метод	Путь	Назначение	Коды ответов
GET	/api/assessment/attempts	История попыток	200, 401, 403, 422
POST	/api/assessment/attempts	Начать тест на грейд	201, 401, 403, 409, 422
GET	/api/assessment/attempts/{attempt_id}	Состояние попытки и текущее задание	200, 401, 403, 404, 422
POST	/api/assessment/attempts/{attempt_id}/abandon	Прервать попытку (засчитывается как неудачная)	200, 401, 403, 404, 409, 422
POST	/api/assessment/attempts/{attempt_id}/answer	Ответить на текущее задание	200, 401, 403, 404, 409, 422
GET	/api/assessment/eligibility	Доступные попытки и правила смены грейда	200, 401, 403, 422
POST	/api/assessment/preview	Как система формирует тест по описанию вакансии	200, 400, 404, 422

Работодатель

Компания, описание потребности, подборки с объяснениями, банк кандидатов.

Метод	Путь	Назначение	Коды ответов
GET	/api/employer/candidates	Поиск по банку кандидатов (порядок — сила подтверждённого профиля)	200, 401, 403, 409, 422
GET	/api/employer/candidates/{candidate_id}	Карточка кандидата (контакты — после принятия приглашения или отклика)	200, 401, 403, 404, 409, 422
GET	/api/employer/candidates/{candidate_id}/profile.pdf	PDF-профиль кандидата	200, 401, 403, 404, 409, 422
GET	/api/employer/candidates/{candidate_id}/test-photo	Кадр с теста с проктором — после принятого приглашения или отклика, при согласии кандидата	200, 401, 403, 404, 409, 422
GET	/api/employer/categories	Категоризированная база: специализация × грейд	200, 401, 403, 422
GET	/api/employer/company	Профиль компании	200, 401, 403, 422
PUT	/api/employer/company	Создать или обновить профиль компании	200, 401, 403, 422
POST	/api/employer/needs/parse	Разобрать описание потребности (NLP): навыки, специализация, грейд	200, 401, 403, 422
GET	/api/employer/searches	Мои подборки	200, 401, 403, 409, 422
POST	/api/employer/searches	Описать потребность и получить подборку	201, 400, 401, 403, 409, 422
DELETE	/api/employer/searches/{search_id}	Удалить подборку	204, 401, 403, 404, 409, 422
GET	/api/employer/searches/{search_id}	Подборка с ранжированием и объяснениями	200, 401, 403, 404, 409, 422
PUT	/api/employer/searches/{search_id}/filters	Уточнить фильтры (новая версия, прежние сохраняются)	200, 401, 403, 404, 409, 422
POST	/api/employer/searches/{search_id}/revert/{version}	Откатить фильтры к версии	200, 401, 403, 404, 409, 422
DELETE	/api/employer/searches/{search_id}/shortlist/{candidate_id}	Убрать из шорт-листа	204, 401, 403, 404, 409, 422
PUT	/api/employer/searches/{search_id}/shortlist/{candidate_id}	Закрепить кандидата в шорт-листе	204, 401, 403, 404, 409, 422

Приглашения

Выход на контакт по инициативе работодателя; статусы; обратная связь рынка.

Метод	Путь	Назначение	Коды ответов
GET	/api/candidate/invitations	Входящие приглашения	200, 401, 403, 422
GET	/api/candidate/invitations/{inv_id}	Открыть приглашение (статус → просмотрено)	200, 401, 403, 404, 422
POST	/api/candidate/invitations/{inv_id}/accept	Принять приглашение — контакты станут видны работодателю	200, 401, 403, 404, 409, 422
POST	/api/candidate/invitations/{inv_id}/decline	Отклонить приглашение с причиной	200, 401, 403, 404, 409, 422
GET	/api/employer/insights	Обратная связь рынка: причины отказов, ожидания по категориям	200, 401, 403, 409, 422
GET	/api/employer/invitations	Отправленные приглашения и их статусы	200, 401, 403, 409, 422
POST	/api/employer/invitations	Пригласить кандидата (вилка обязательна)	201, 401, 403, 404, 409, 422, 429
POST	/api/employer/invitations/{inv_id}/withdraw	Отозвать приглашение	200, 401, 403, 404, 409, 422
GET	/api/employer/salary-check	Индикатор вилки до отправки приглашения	200, 401, 403, 404, 409, 422

Вакансии и отклики

Классический сценарий: публикация вакансий и самостоятельный отклик.

Метод	Путь	Назначение	Коды ответов
`GET`	`/api/candidate/applications`	Мои отклики и их статусы	200, 401, 403, 422
`POST`	`/api/employer/applications/{app_id}/status`	Сменить статус отклика	200, 401, 403, 404, 409, 422
`GET`	`/api/employer/vacancies`	Мои вакансии	200, 401, 403, 409, 422
`POST`	`/api/employer/vacancies`	Опубликовать вакансию (вилка обязательна)	201, 400, 401, 403, 409, 422
`PUT`	`/api/employer/vacancies/{vacancy_id}`	Редактировать вакансию	200, 400, 401, 403, 404, 409, 422
`GET`	`/api/employer/vacancies/{vacancy_id}/applications`	Отклики на вакансию	200, 401, 403, 404, 409, 422
`GET`	`/api/vacancies`	Лента вакансий (сортировка по соответствию профилю)	200, 401, 403, 422
`GET`	`/api/vacancies/{vacancy_id}`	Вакансия	200, 401, 403, 404, 422
`POST`	`/api/vacancies/{vacancy_id}/apply`	Откликнуться (контакты станут видны работодателю)	201, 401, 403, 404, 409, 422

Регулярные задания

Короткие задания от работодателей для актуальности профиля.

Метод	Путь	Назначение	Коды ответов
`GET`	`/api/candidate/tasks`	Мои регулярные задания (новое назначается раз в 2 недели)	200, 401, 403, 422
`POST`	`/api/candidate/tasks/{assignment_id}/submit`	Отправить решение	200, 400, 401, 403, 404, 409, 422
`POST`	`/api/employer/submissions/{assignment_id}/review`	Оценить решение по рубрике	200, 401, 403, 404, 409, 422
`GET`	`/api/employer/tasks`	Мои задания	200, 401, 403, 409, 422
`POST`	`/api/employer/tasks`	Создать регулярное задание	201, 401, 403, 409, 422
`GET`	`/api/employer/tasks/{task_id}/submissions`	Решения кандидатов	200, 401, 403, 404, 409, 422

ФСП

Привязка ФСП ID (OIDC) и синхронизация достижений из реестра ФСП.

Метод	Путь	Назначение	Коды ответов
`DELETE`	`/api/fsp/link`	Отвязать ФСП ID	200, 401, 403, 422
`GET`	`/api/fsp/link/callback`	Callback ФСП ID (браузерный редирект)	302, 422
`GET`	`/api/fsp/link/start`	Начать привязку ФСП ID (редирект на страницу входа ФСП ID)	200, 401, 403, 422
`POST`	`/api/fsp/sync`	Обновить достижения из реестра ФСП	200, 401, 403, 409, 422, 503

Примеры

Начать тест и ответить на задание:

```
POST /api/assessment/attempts
{"grade": "middle"}

→ 201 {"id": "…", "status": "in_progress", "item": {"position": 0, "kind": "text", "prompt": "Что выведет программа?…", "code":
"print(-30 // 7, …)", "time_limit": 150}}

POST /api/assessment/attempts/{id}/answer
{"position": 0, "value": "-5 5 -4", "time_ms": 41000, "tab_switches": 0}
```

Описать потребность и получить подборку:

```
POST /api/employer/searches
{"title": "Python-разработчик", "text": "Нужен бэкенд: Python, FastAPI, PostgreSQL. Опыт от 2 лет. Вилка 250-320 тыс ₽"}

→ 201 {"search": {"category": "Backend · Middle", …}, "categories": […], "results": [{"card": {"alias": "Backend #6306", "contacts":
null, …},
        "score": 0.70, "components": {"need_fit": 0.23, …}, "reasons": [{"kind": "positive", "text": "Подтверждено тестом: Python,
PostgreSQL"}, …]}]}
```

Пригласить кандидата (вилка обязательна):

```
POST /api/employer/invitations
{"candidate_id": "…", "title": "Python-разработчик", "description": "…", "salary_from": 250000, "salary_to": 320000,
 "contact_method": "telegram", "contact_value": "@hr_company"}
```

Используемые библиотеки и компоненты

> Сгенерировано `docs/tools/gen_docs.py` из `uv.lock` и `frontend/package-lock.json`. Все компоненты — открытое ПО (MIT / Apache-2.0 / BSD / ISC / OFL / LGPL с динамической линковкой).

Инфраструктура

Компонент	Версия	Лицензия	Назначение
Python	3.12 (`python:3.12-slim`)	PSF	Образ бэкенда и реестра ФСП
uv	0.5 (`ghcr.io/astral-sh/uv:0.5`)	Apache-2.0 / MIT	Установка зависимостей по `uv.lock`
PostgreSQL	16 (`postgres:16-alpine`)	PostgreSQL License	Основная БД, БД Keycloak
Keycloak	26.0 (`quay.io/keycloak/keycloak:26.0`)	Apache-2.0	Аутентификация, подтверждение email; имитация ФСП ID
Mailpit	1.31 (`axllent/mailpit:v1.31`)	MIT	Перехват писем в демо (подтверждение email, уведомления)
nginx	1.27 (`nginx:1.27-alpine`)	BSD-2	Раздача фронтенда, прокси `/api`
Node.js	22 (`node:22-alpine`, сборка)	MIT	Сборка фронтенда
Docker Compose	v2	Apache-2.0	Оркестрация стека
JetBrains Mono	2.304	OFL-1.1	Фирменный шрифт ФСП (PDF, интерфейс)
rubert-tiny2 (cointegrated)	—	MIT	Локальные эмбеддинги (опционально, `WITH_EMBEDDINGS=1`)
bge-reranker-v2-m3 (BAAI)	—	Apache-2.0	Кросс-энкодер гибридного подбора (опционально, `FSP_RANKER=hybrid`)
Модели Human (BlazeFace, FaceRes, liveness/antispoof, CenterNet)	из `@vladmandic/human`	MIT	Цифровой проктор, исполняются в браузере кандидата

Python (бэкенд, ML-ядро)

Пакет	Версия	Лицензия	Назначение
fastapi	0.142.2	MIT	веб-фреймворк API, OpenAPI
uvicorn	0.54.0	BSD-3	ASGI-сервер
sqlalchemy	2.1.3	MIT	ORM
alembic	1.20.0	MIT	миграции БД
psycopg	3.3.6	LGPL-3 (динамическая линковка)	драйвер PostgreSQL
pydantic	2.13.5	MIT	валидация и схемы
pydantic-settings	2.15.0	MIT	конфигурация из окружения
pyjwt	2.15.1	MIT	проверка JWT (OIDC)
httpx	0.28.1	BSD-3	HTTP-клиент (реестр ФСП, Keycloak, webhook)
python-multipart	0.0.32	Apache-2.0	загрузка файлов
fpdf2	2.8.9	LGPL-3	PDF-профиль, синтетические резюме
email-validator	2.3.0	Unlicense	валидация email
numpy	2.5.3	BSD-3	IRT/CAT, ранжирование
scipy	1.18.1	BSD-3	оптимизация в MML-EM
pyyaml	6.0.3	MIT	справочники
rapidfuzz	3.14.6	MIT	нечёткий поиск технологий
scikit-learn	1.9.1	BSD-3	TF-IDF эмбеддинги
lightgbm	4.7.0	MIT	LTR-модель подбора (эксперимент бенчмарка)
cryptography	50.0.2	Apache-2.0 / BSD	AES-256-GCM для эталонов лица (проктор)
sentence-transformers	6.1.0	Apache-2.0	эмбеддинги rubert-tiny2 и кросс-энкодер (опционально)
pdfplumber	0.11.10	MIT	извлечение текста из PDF
pandas	3.0.6	BSD-3	задания по pandas (исполнение сниппетов)
matplotlib	3.11.2	PSF-based	графики валидации
pytest	9.1.1	MIT	тесты
ruff	0.16.10	MIT	линтер/форматтер
markdown	3.11	BSD-3	сборка PDF-документации

Фронтенд (npm)

Пакет	Версия	Лицензия	Назначение
@fontsource-variable/inter	5.3.0	OFL-1.1	шрифт текста
@fontsource/jetbrains-mono	5.3.0	OFL-1.1	фирменный шрифт ФСП
@tanstack/react-query	5.104.1	MIT	кеш запросов к API
@vladmandic/human	3.3.6	MIT	цифровой проктор в браузере: лицо, телефон, эмбеддинг, liveness
clsx	2.1.1	MIT	классы
keycloak-js	26.2.4	Apache-2.0	OIDC-клиент (PKCE)
lucide-react	1.52.0	ISC	иконки
react	19.3.0	MIT	UI
react-dom	19.3.0	MIT	UI
react-router-dom	7.18.4	MIT	маршрутизация
@tailwindcss/vite	4.3.3	MIT	сборка стилей
@types/node	24.19.1	MIT	типы
@types/react	19.3.0	MIT	типы
@types/react-dom	19.3.0	MIT	типы
@vitejs/plugin-react	6.1.1	MIT	сборка React
oxlint	1.86.0	MIT	линтер
tailwindcss	4.3.3	MIT	стили
typescript	6.0.3	Apache-2.0	типизация
vite	8.3.2	MIT	сборка фронтенда

По умолчанию все модели работают локально на CPU, внешние сервисы не вызываются. Опционально (`LLM_ENABLED=1` , по умолчанию выключено) — Yandex AI Studio в контуре РФ как запасной разбор опыта в резюме; логирование запросов на стороне Яндекса отключено (152-ФЗ).

Сборка, развёртывание и локальный запуск

1. Быстрый старт (Docker Compose, ~5 минут)

Требования: Docker 24+ с Compose v2, 4 ГБ свободной RAM, свободные порты 80, 8000, 8025, 8080, 8090.

```
git clone https://github.com/Izymka/fsp-job-board.git && cd fsp-job-board
cp .env.example .env          # для локального запуска можно ничего не менять
docker compose up -d --build  # первая сборка ~3-5 мин
docker compose ps             # дождитесь healthy у keycloak и backend (~1-2 мин)
```

Адрес	Что там
http://localhost	Платформа
http://localhost/api/docs	Swagger UI (OpenAPI)
http://localhost:8080	Keycloak (админ: `admin` / `admin` из `.env`)
http://localhost:8025	Mailpit — письма подтверждения email и уведомления
http://localhost:8090/docs	Реестр ФСП (имитация)

Удобнее через `make up`: дополнительно разрешает HTTP для админки Keycloak на демо-стенде (realm `master`; realm-ы платформы уже настроены на `sslRequired: none` — в продакшене включите TLS на прокси и `sslRequired: external`).

При старте бэкенд применяет миграции (`alembic upgrade head`) и загружает демо-данные (`python -m app.seed`): ~300 кандидатов, прошедших симулированный тест, 7 компаний с вакансиями, регулярные задания, историю приглашений.

Демо-аккаунты (тестовые учётные данные стенда)

Логин	Пароль	Роль
`employer@demo.fsp`	`demo12345`	Работодатель, компания «Рубль+»
`maria@demo.fsp`	`demo12345`	Кандидат Backend · Middle с привязанным ФСП ID
`candidate@demo.fsp`	`demo12345`	Новый кандидат — для показа онбординга
ФСП ID: `fsp-000123`	`demo12345`	Участник ФСП с достижениями (для «Привязать ФСП ID»)
ФСП ID: `fsp-000999`	`demo12345`	Участник без истории ФСП

Новая регистрация: «Регистрация» → форма Keycloak → письмо в Mailpit (http://localhost:8025) → ссылка подтверждения → выбор роли.

2. Сценарий проверки (ТЗ §3.3)

1. **Кандидат:** войти `candidate@demo.fsp` → роль «Ищу работу» + согласие → опрос (Backend, стек, Middle, ожидания) → «Тест и грейд» → пройти тест → категория «Backend · Middle» (или тест на уровень ниже).
2. **Работодатель:** войти `employer@demo.fsp` → «Подборки» → «Новая подборка» → вставить текст потребности (кнопка «Вставить пример») → «Показать подборку» → рекомендованные категории, кандидаты с объяснением → фильтр «есть ФСП» → «Сбросить уточнения» → «Пригласить» (индикатор вилки).
3. **Контакт:** войти `maria@demo.fsp` или кандидатом из шага 1 → «Приглашения» → открыть (статус «просмотрено») → «Принять» → у работодателя в «Приглашения» появились ФИО и контакты.
4. **Отклик:** кандидат → «Вакансии» → откликнуться → работодатель → «Вакансии» → «Отклики» (контакты открыты сразу).
5. **Механики:** публичные страницы «Тест из описания вакансии» (`/test-demo`) и «Механика подбора и тестирования» (`/how-it-works`).

3. Развёртывание на сервере

VPS: 2–4 vCPU, 4–8 ГБ RAM, Ubuntu 22.04+, Docker. Домены, например `jobs.example.ru` (платформа) и `auth.example.ru` (Keycloak).

1. В `.env`: `PUBLIC_URL=https://jobs.example.ru`, `KEYCLOAK_PUBLIC_URL=https://auth.example.ru`, сменить `POSTGRES_PASSWORD`, `KEYCLOAK_ADMIN_PASSWORD`, `ASSESSMENT_SECRET`, `FSP_ID_CLIENT_SECRET`, `FSP_REGISTRY_API_KEY`.
2. В `keycloak/realm-*.json` добавить redirect URI для домена (`https://jobs.example.ru/*`, `https://jobs.example.ru/api/fsp/link/callback`) и настроить SMTP реального почтового сервера; удалить демо-пользователей.
3. TLS — обратный прокси (Caddy/nginx) перед `frontend:80` и `keycloak:8080`. Пример Caddyfile: `jobs.example.ru { reverse_proxy localhost:80 } auth.example.ru { reverse_proxy localhost:8080 }`
4. `docker compose up -d --build`. Для Keycloak в продакшене используйте `start` вместо `start-dev` и `KC_PROXY_HEADERS=xforwarded`.
5. Резервное копирование: `docker compose exec postgres pg_dump -U fsp fsp_jobs > backup.sql`.

Опционально `WITH_EMBEDDINGS=1` — бэкенд собирается с нейросетевыми эмбеддингами rubert-tiny2 (CPU, +~700 МБ к образу, ~150 МБ RAM).

Диагностика

- `docker compose logs -f backend keycloak` — логи; у бэкенда в начале должно быть «Готово: 300 кандидатов…».
- Страница Keycloak «Требуется HTTPS» — выполните `make kc-demo` (или задайте `sslRequired` в realm).
- `docker pull` / сборка «висит» на загрузке метаданных образа в Docker Desktop — зависает помощник учётных данных (`credsStore: desktop` в `~/.docker/config.json`): перезапустите Docker Desktop или соберите с чистым конфигом `DOCKER_CONFIG=$(mktemp -d) docker compose`

`build`.

4. Локальная разработка без Docker

Требования: Python 3.12, uv, Node.js 20.19+ или 22.12+.

```
uv sync --all-packages          # Python-зависимости всех модулей
make dev                        # всё сразу: реестр, API и фронтенд в одном терминале (Ctrl+C — остановить)
# или по отдельности, каждый в своём терминале:
make dev-registry               # реестр ФСП на :8090 (отдельный терминал)
make dev-backend                # API на :8000: SQLite, AUTH_MODE=dev (вход без Keycloak), seed
make dev-frontend               # фронтенд на :5173 (прокси /api → :8000)
```

В режиме `AUTH_MODE=dev` страница входа предлагает демо-аккаунты без пароля — **только для локальной отладки**; привязка ФСП ID в этом режиме — демо-кнопкой по номеру.

5. Тесты, линтер, валидация

```
make test        # 607 тестов ML-ядра + 19 сквозных тестов API + сборка фронтенда
make lint        # ruff
make validate    # процедура валидации → docs/validation.md, docs/img/*.png (~2 мин)
uv run --with markdown python docs/build_pdf.py   # документация одним PDF → docs/fsp-job-board-docs.pdf
```

Концепции развития: защита от недобросовестных работодателей и интеграция с ATS

ТЗ относит эти механизмы к развитию продукта («будет плюсом»): ожидается концепция и архитектурный подход. Часть основы уже реализована в MVP — отмечено ✅.

1. Защита от фиктивных вакансий и недобросовестных работодателей

Принципы

- Кандидат должен видеть, кто ему пишет, *до* раскрытия своих данных.
- Каждое действие работодателя оставляет измеримый след, из которого строится репутация.
- Санкции градуированы: от предупреждения до блокировки, с апелляцией.

Слои защиты

Слой	Механизм	Статус
Прозрачность условий	Вилка «от–до» обязательна в вакансии и приглашении (валидация API + CHECK в БД)	✅
Защита ПДн кандидата	Контакты открываются только после принятия приглашения / отклика; журнал раскрытия	✅
Ограничение рассылок	Квота приглашений в неделю; запрет повторного приглашения при активном	✅
Репутация	Кандидат видит у компании: приглашений за 90 дней, долю принятых, долю отвеченных откликов	✅
Сигнал от кандидатов	Причины отказов (в т.ч. «компания/отрасль») агрегируются	✅
Верификация юрлица	Проверка ИНН/ОГРН по ЕГРЮЛ (API ФНС), статус «действующее», совпадение названия; поле `inn` уже есть	концепция
Подтверждение домена	Письмо на корпоративный адрес домена сайта компании; бейдж «домен подтверждён»	концепция
Модерация по риску	Автоматические флаги: массовые приглашения с вилкой сильно ниже рынка категории (по данным «обратной связи рынка»), всплеск отказов «компания», жалобы, свежая регистрация + высокая активность → ручная проверка, временная заморозка квоты	концепция
Жалобы	Кнопка «пожаловаться» в приглашении с типами (фейковая вакансия, просьба оплатить обучение, несоответствие условий)	концепция
Анти-скам паттерны	Детекция в тексте приглашения: просьбы предоплаты, «обучение за свой счёт», внешние ссылки на сбор данных	концепция
Связь с ФСП	Партнёры ФСП получают бейдж «партнёр Федерации» по реестру партнёров	концепция

Модель репутации

`trust = f(верификация юрлица, подтверждение домена, acceptance rate, response rate к откликам, доля отказов «компания», жалобы, возраст аккаунта)` — рассчитывается ежедневно; при низком доверии: сниженная квота, пометка «новая компания», обязательная проверка вакансий модератором.

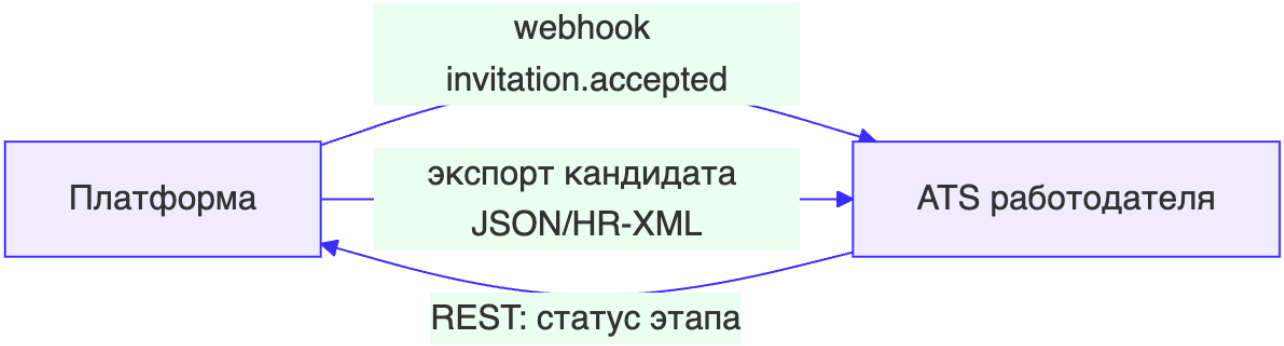
2. Интеграция с ATS работодателя

Что уже есть ✅

- **Webhook** `invitation.accepted` на URL из профиля компании: JSON с позицией, вилкой и контактами кандидата (только после его согласия — принятия приглашения).
- **REST API** с OpenAPI-описанием: подборки, кандидаты, приглашения, отклики — интегратор может работать от имени сервисного пользователя работодателя.
- **PDF-профиль кандидата** в стандартизированном виде.

Развитие

1. **Подпись webhook** (HMAC-SHA256 в заголовке `X-FSP-Signature`), ретраи с экспоненциальной задержкой, журнал доставок, события `application.created`, `invitation.declined`, `candidate.updated`.
2. **Сервисные клиенты** (OAuth2 client credentials в Keycloak) со scope `jobs:read`, `invitations:write`.
3. **Экспорт кандидата** в форматах JSON Resume / HR-Open (HR-XML) — с обезличиванием до согласия.
4. **Коннекторы** для популярных российских ATS (Huntflow, Talantix, E-Staff, Поток): маппинг кандидата и статусов, двусторонняя синхронизация этапов (статус в ATS «интервью» → статус отклика «интервью» на платформе, кандидат видит прогресс).
5. **Импорт вакансий** из ATS с обязательной проверкой наличия вилки.


webhook
invitation.accepted
Платформа
экспорт кандидата
JSON/HR-XML
ATS работодателя
REST: статус этапа

Протокол живого пилота тестирования

Синтетическая валидация (validation.md) проверяет механику при известных допущениях. Живой пилот проверяет сами допущения: реальную сложность семейств, время ответа, согласие грейда с экспертной оценкой.

Участники

15–20 человек: команда, знакомые разработчики и аналитики разных грейдов (минимум по 3 на Junior / Middle / Senior), желательно 2–3 участника соревнований ФСП.

Процедура

1. **Экспертная оценка до теста.** Два эксперта независимо оценивают грейд участника по короткому структурированному интервью (15 минут, единый опросник по специализации) — эталон для сравнения. Эксперты не видят результатов теста.
2. **Тест.** Участник проходит опрос и тест на самооценённый грейд на стенде (обычный пользовательский путь).
3. **Ретест.** Через 1–3 дня — повторный тест (другие варианты заданий) на тот же грейд; регистрируется решение и θ.
4. **Анкета UX** (5 вопросов, шкала 1–5): понятность заданий, достаточность времени, ощущение честности, понятность результата, желание пройти повышение.

Метрики

- Согласие грейда теста с экспертным: доля совпадений, ±1 ступень, Cohen's κ; согласие двух экспертов между собой (эталон «человеческой» надёжности).
- Тест–ретест: совпадение решения, корреляция θ.
- По семействам: доля верных ответов, точечно-бисериальная корреляция, медианное время — семейства с аномалиями идут на доработку; ответы пилота — первые данные для перекалибровки (MML-EM).
- Технические: число заданий до решения, длительность теста, ошибки интерфейса.

Обработка данных

Участники дают согласие на обработку ПДн; в отчёт идут только обезличенные агрегаты. Результаты — раздел «Живой пилот» в validation.md.